
NAMING

Names play an important role in all computer systems. They are used to share resources, to uniquely identify entities, to refer to locations, and more. An important issue with naming is that a name can be resolved to the entity it refers to. Name resolution thus allows a process to access the named entity. To resolve names, it is necessary to implement a naming system. The difference between naming in distributed systems and nondistributed systems lies in the way naming systems are implemented.

In a distributed system, the implementation of a naming system is itself often distributed across multiple machines. How this distribution is done plays a key role in the efficiency and scalability of the naming system. In this chapter, we concentrate on three different, important ways that names are used in distributed systems.

First, we consider so-called flat-naming systems. In such systems, entities are referred to by an identifier that, in principle, has no meaning at all. In addition, flat names bare no structure, implying that we need special mechanisms to trace the location of such entities. We discuss various approaches, ranging from chains of forwarding links, to distributed hash tables, to hierarchical location services.

In practice, humans prefer to use readable names. Such names are often structured, as is well known from the way Web pages are referred to. Structured names allow for a highly systematic way of finding the server responsible for the named entity, as exemplified by the Domain Name System. We discuss the general principles, as well as scalability issues.

Finally, humans often prefer to describe entities by means of various characteristics, leading to a situation in which we need to resolve a description by means of the attributes assigned to an entity. As we shall see, this type of name resolution is notoriously difficult, especially in combination with searching.

5.1 Names, identifiers, and addresses

Let us start by taking a closer look at what a name actually is. A name in a distributed system is a string of bits or characters that is used to refer to an entity. An entity in a distributed system can be practically anything. Typical examples include resources such as hosts, printers, disks, and files. Other well-known examples of entities that are often explicitly named are processes, users, mailboxes, newsgroups, Web pages, graphical windows, messages, network connections, and so on.

Entities can be operated on. For example, a resource such as a printer offers an interface containing operations for printing a document, requesting the status of a print job, and the like. Furthermore, an entity such as a network connection may provide operations for sending and receiving data, setting quality-of-service parameters, requesting the status, and so forth.

To operate on an entity, it is necessary to access it, for which we need an **access point**. An access point is yet another, but special, kind of entity in a distributed system. The name of an access point is called an **address**. The address of an access point of an entity is also simply called an address of that entity.

An entity can offer more than one access point. As a comparison, a telephone can be viewed as an access point of a person, whereas the telephone number corresponds to an address. Indeed, many people nowadays have several telephone numbers, each number corresponding to a point where they can be reached. In a distributed system, a typical example of an access point is a host running a specific server, with its address formed by the combination of, for example, an IP address and port number (i.e., the server's transport-level address).

An entity may change its access points in the course of time. For example, when a mobile computer moves to another location, it is often assigned a different IP address than the one it had before. Likewise, when a person moves to another city or country, it is often necessary to change telephone numbers as well. In a similar fashion, changing jobs or Internet Service Providers, means changing your e-mail address.

An address is thus just a special kind of name: it refers to an access point of an entity. Because an access point is tightly associated with an entity, it would seem convenient to use the address of an access point as a regular name for the associated entity. Nevertheless, this is hardly ever done as such naming is generally very inflexible and often human unfriendly.

For example, it is not uncommon to regularly reorganize a distributed system so that a specific server is now running on a different host than previously. The old machine on which the server used to be running may be reassigned to a completely different server. In other words, an entity may easily change an access point, or an access point may be reassigned to a different entity. If an address is used to refer to an entity, we will have

an invalid reference the instant the access point changes or is reassigned to another entity. Therefore, it is much better to let a service be known by a separate name independent of the address of the associated server.

Likewise, if an entity offers more than one access point, it is not clear which address to use as a reference. For instance, many organizations distribute their Web service across several servers. If we would use the addresses of those servers as a reference for the Web service, it is not obvious which address should be chosen as the best one. Again, a much better solution is to have a single name for the Web service independent from the addresses of the different Web servers.

These examples illustrate that a name for an entity that is independent from its addresses is often much easier and more flexible to use. Such a name is called **location independent**.

In addition to addresses, there are other types of names that deserve special treatment, such as names that are used to uniquely identify an entity. A true **identifier** is a name that has the following properties [Wieringa and de Jonge, 1995]:

1. An identifier refers to at most one entity.
2. Each entity is referred to by at most one identifier.
3. An identifier always refers to the same entity (i.e., it is never reused).

By using identifiers, it becomes much easier to unambiguously refer to an entity. For example, assume two processes each refer to an entity by means of an identifier. To check if the processes are referring to the same entity, it is sufficient to test if the two identifiers are equal. Such a test would not be sufficient if the two processes were using regular, nonunique, nonidentifying names. For example, the name "John Smith" cannot be taken as a unique reference to just a single person.

Likewise, if an address can be reassigned to a different entity, we cannot use an address as an identifier. Consider the use of telephone numbers, which are reasonably stable in the sense that a telephone number will often for some time refer to the same person or organization. However, using a telephone number as an identifier will not work, as it can be reassigned in the course of time. Consequently, Bob's new bakery may be receiving phone calls for Alice's old antique store for a long time. In this case, it would have been better to use a true identifier for Alice instead of her phone number.

Addresses and identifiers are two important types of names that are each used for very different purposes. In many computer systems, addresses and identifiers are represented in machine-readable form only, that is, in the form of bit strings. For example, an Ethernet address is essentially a random string of 48 bits. Likewise, memory addresses are typically represented as 32-bit or 64-bit strings.

Another important type of name is that which is tailored to be used by humans, also referred to as **human-friendly names**. In contrast to addresses and identifiers, a human-friendly name is generally represented as a character string. These names appear in many different forms. For example, files in Unix systems have character-string names that can generally be as long as 255 characters, and which are defined entirely by the user. Similarly, DNS names are represented as relatively simple case-insensitive character strings.

Having names, identifiers, and addresses brings us to the central theme of this chapter: how do we resolve names and identifiers to addresses? Before we go into various solutions, it is important to realize that there is often a close relationship between name resolution in distributed systems and message routing [Shoch, 1978]. In principle, a naming system maintains a **name-to-address binding** which in its simplest form is just a table of *(name, address)* pairs. However, in distributed systems that span large networks and for which many resources need to be named, a centralized table is not going to work.

Instead, what often happens is that a name is decomposed into several parts such as ftp.cs.vu.nl. and that name resolution takes place through a recursive lookup of those parts. For example, a client needing to know the address of the FTP server named by ftp.cs.vu.nl. would first resolve nl to find the server NS(nl) responsible for names that end with nl, after which the rest of the name is passed to server NS(nl). This server may then resolve the name vu to the server NS(vu.nl) responsible for names that end with vu.nl. who can further handle the remaining name ftp.cs. Eventually, this leads to routing the name resolution request as:

$$\text{NS}(.) \Rightarrow \text{NS}(\text{nl}) \Rightarrow \text{NS}(\text{vu.nl}) \Rightarrow \text{address of ftp.cs.vu.nl}$$

where NS(.) denotes the server that can return the address of NS(nl), also known as the **root server**. NS(vu.nl) will return the actual address of the FTP server. It is interesting to note that the boundaries between name resolution and message routing are starting to blur.

Note 5.1 (More information: Information-centric networking)

Name resolution and message routing play a central role in **information-centric networking**, or simply **ICN** [Ahlgren et al., 2012]. This type of networking revolves around the principle that applications are not really interested to know *where* an entity is stored, but rather that they can get a copy in order to access it locally when needed. To this end, much research has been spent since approximately 2007 on designing an alternative to the host-based addressing schemes that are common in today's Internet. In particular, the main idea is that an application can retrieve an entity from the network by using that entity's name. The network takes that name as input, and subsequently routes a request to an appropriate location where the entity is stored, to return a copy to the requester.

Key to the success of this approach is, of course, a form of name-based routing, which is essentially a means for resolving a name to an address where the associated entity is to be found. How that routing takes place depends on the organization of names. Flat names may require solutions such as those for distributed hash tables, which we discuss in Section 5.2. Structured names can be efficiently resolved using hierarchical solutions as we discuss in Section 5.3.

Note 5.2 (More information: Self-certifying names)

If a name is being used to refer to an entity, how do we know that we are actually accessing the intended entity? In other words, how can we ensure that an entity and its name are uniquely bound to each other? In the case of immutable data objects, there is a simple solution: take a *hash* of the data object and use that hash value as its name. When the data object is retrieved, the receiving process can hash the object and see whether the hash value corresponds to the one it used as the object's name. This is a simple version of a **self-certifying name**, which is a name that can be checked locally to see if it truly belongs to the entity it refers to. We return to secure naming in Section 9.4.

In the following sections we will consider three different classes of naming systems. First, we will take a look at how identifiers can be resolved to addresses. In this case, we will also see an example where name resolution is actually indistinguishable from message routing. After that, we consider human-friendly names and descriptive names (i.e., entities that are described by a collection of names).

5.2 Flat naming

Above, we explained that identifiers are convenient to uniquely represent entities. In many cases, identifiers are simply random bit strings, which we conveniently refer to as unstructured, or flat names. An important property of such a name is that it does not contain any information whatsoever on how to locate the access point of its associated entity. In the following, we will take a look at how flat names can be resolved, or, equivalently, how we can locate an entity when given only its identifier.

Simple solutions

We first consider two simple solutions for locating an entity: broadcasting and forwarding pointers. Both solutions are mainly applicable only to local-area networks. Nevertheless, in that environment, they often do the job well, making their simplicity particularly attractive. However, the use of broadcasting

and forwarding pointers imposes scalability problems. Broadcasting or multicasting is difficult to implement efficiently in large-scale networks whereas long chains of forwarding pointers introduce performance problems and are susceptible to broken links.

Broadcasting

Consider a distributed system built on a computer network that offers efficient broadcasting facilities. Typically, such facilities are offered by local-area networks in which all machines are connected to a single cable or the logical equivalent thereof. Also, local-area wireless networks fall into this category.

Locating an entity in such an environment is simple: a message containing the identifier of the entity is broadcast to each machine and each machine is requested to check whether it has that entity. Only the machines that can offer an access point for the entity send a reply message containing the address of that access point.

This principle is used in the Internet **address resolution protocol (ARP)** to find the data-link address of a machine when given only an IP address [Plummer, 1982]. In essence, a machine broadcasts a packet on the local network asking who is the owner of a given IP address. When the message arrives at a machine, the receiver checks whether it should listen to the requested IP address. If so, it sends a reply packet containing, for example, its Ethernet address.

Broadcasting becomes inefficient when the network grows. Not only is network bandwidth wasted by request messages, but, more seriously, too many hosts may be interrupted by requests they cannot answer. One possible solution is to switch to multicasting, by which only a restricted group of hosts receives the request. For example, Ethernet networks support data-link level multicasting directly in hardware.

Multicasting can also be used to locate entities in point-to-point networks. For example, the Internet supports network-level multicasting by allowing hosts to join a specific multicast group. Such groups are identified by a multicast address. When a host sends a message to a multicast address, the network layer provides a best-effort service to deliver that message to all group members. Efficient implementations for multicasting in the Internet are discussed in Deering and Cheriton [1990] and Deering et al. [1996].

A multicast address can be used as a general location service for multiple entities. For example, consider an organization where each employee has his or her own mobile computer. When such a computer connects to the locally available network, it is dynamically assigned an IP address. In addition, it joins a specific multicast group. When a process wants to locate computer A, it sends a “where is A?” request to the multicast group. If A is connected, it responds with its current IP address.

Another way to use a multicast address is to associate it with a replicated entity, and to use multicasting to locate the *nearest* replica. When sending a request to the multicast address, each replica responds with its current (normal) IP address. A crude way to select the nearest replica is to choose the one whose reply comes in first, but as it turns out, selecting a nearest replica is generally not that easy.

Forwarding pointers

Another popular approach to locating mobile entities is to make use of forwarding pointers [Fowler, 1985]. The principle is simple: when an entity moves from A to B, it leaves behind in A a reference to its new location at B. The main advantage of this approach is its simplicity: as soon as an entity has been located, for example by using a traditional naming service, a client can look up the current address by following the chain of forwarding pointers.

There are also drawbacks. First, if no special measures are taken, a chain for a highly mobile entity can become so long that locating that entity is prohibitively expensive. Second, all intermediate locations in a chain will have to maintain their part of the chain of forwarding pointers as long as needed. A third (and related) drawback is the vulnerability to broken links. As soon as any forwarding pointer is lost, the entity can no longer be reached. An important issue is, therefore, to keep chains relatively short, and to ensure that forwarding pointers are robust.

Note 5.3 (More information: SSP chains)

To better understand how forwarding pointers work, consider their use with respect to remote objects: objects that can be accessed by means of a remote procedure call. Following the approach in **SSP chains** [Shapiro et al., 1992], each forwarding pointer is implemented as a (*client stub*, *server stub*) pair as shown in Figure 5.1 (We note that in the original terminology, a server stub was called a scion, leading to (*stub*, *scion*) pairs, which explains its name.) A server stub contains either a local reference to the actual object or a local reference to a remote client stub for that object.

Whenever an object moves from address space A to B, it leaves behind a client stub in its place in A and installs a server stub that refers to the client stub in B. An interesting aspect of this approach is that migration is completely transparent to a client. The only thing the client sees of an object is a client stub. How, and to which location that client stub forwards its invocations, are hidden from the client. Also note that this use of forwarding pointers is not like looking up an address. Instead, a client's request is forwarded along the chain to the actual object.

Now suppose that process P_1 in Figure 5.1 passes its reference to process P_2 . Reference passing is done by installing a copy p' of client stub p in the address space of process P_2 . Client stub p' refers to the same server stub as p , so that the forwarding invocation mechanism works the same as before.

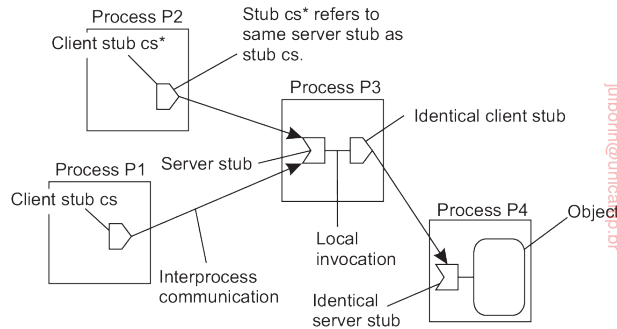


Figure 5.1: The principle of forwarding pointers using *(client stub, server stub)* pairs.

To shortcut a chain of *(client stub, server stub)* pairs, an object invocation carries the identification of the client stub from where that invocation was initiated. A client-stub identification consists of the client's transport-level address, combined with a locally generated number to identify that stub. When the invocation reaches the object at its current location, a response is sent back to the client stub where the invocation was initiated (often without going back up the chain). The current location is piggybacked with this response, and the client stub adjusts its companion server stub to the one in the object's current location. This principle is shown in Figure 5.2.

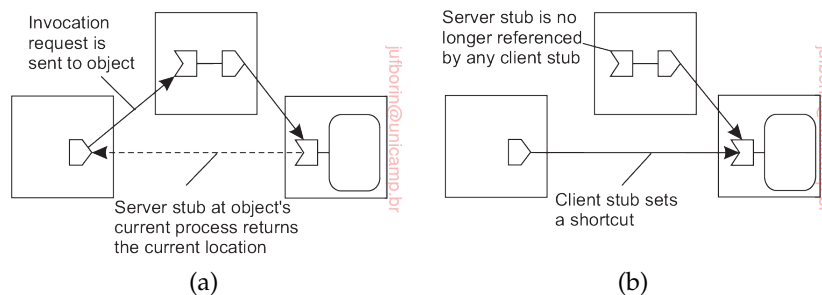


Figure 5.2: Redirecting a forwarding pointer by storing a shortcut in a client stub.

There is a trade-off between sending the response directly to the initiating client stub, or along the reverse path of forwarding pointers. In the former case, communication is faster because fewer processes may need to be passed. On the other hand, only the initiating client stub can be adjusted, whereas sending the response along the reverse path allows adjustment of all intermediate stubs.

When a server stub is no longer referred to by any client, it can be removed. This by itself is strongly related to distributed garbage collection, a generally far from trivial problem, to which we refer the interested reader to [Abdullahi and Ringwood, 1998], [Plainfosse and Shapiro, 1995], and [Veiga and Ferreira, 2005].

Problems arise when a process in a chain of *(client stub, server stub)* pairs

crashes or becomes otherwise unreachable. Several solutions are possible. One possibility, as followed in Emerald [Jul et al., 1988], and in the LII system [Black and Artsy, 1990], is to let the machine where an object was created (called the object's **home location**), always keep a reference to its current location. That reference is stored and maintained in a fault-tolerant way. When a chain is broken, the object's home location is asked where the object is now. To allow an object's home location to change, a traditional naming service can be used to record the current home location.

Home-based approaches

A popular approach to supporting mobile entities in large-scale networks is to introduce a **home location**, which keeps track of the current location of an entity. Special techniques may be applied to safeguard against network or process failures. In practice, the home location is often chosen to be the place where an entity was created.

The home-based approach is used as a fall-back mechanism for location services based on forwarding pointers. Another example where the home-based approach is followed is in Mobile IP [Johnson et al., 2004], which we briefly explained in Note 3.9. Each mobile host uses a fixed IP address. All communication to that IP address is initially directed to the mobile host's **home agent**. This home agent is located on the local-area network corresponding to the network address contained in the mobile host's IP address. In the case of IPv6, it is realized as a network-layer component. Whenever the mobile host moves to another network, it requests a temporary address that it can use for communication. This **care-of address** is registered at the home agent.

When the home agent receives a packet for the mobile host, it looks up the host's current location. If the host is on the current local network, the packet is simply forwarded. Otherwise, it is tunneled to the host's current location, that is, wrapped as data in an IP packet and sent to the care-of address. At the same time, the sender of the packet is informed of the host's current location. This principle is shown in Figure 5.3. Note that the IP address is effectively used as an identifier for the mobile host.

An important aspect is that this whole mechanism is largely hidden for applications. In other words, the original IP address associated with the mobile host can be used by an application without further ado. Client-side software that is part of the application-independent communication layer will handle the redirection to the target's current location. Likewise, at the target's location, a message that has been tunneled will be unpacked and handed to the application on the mobile host as if it were using its original address. Indeed, Mobile IP establishes a high degree of location transparency.

Figure 5.3 also illustrates a drawback of home-based approaches in large-scale networks. To communicate with a mobile entity, a client first has to

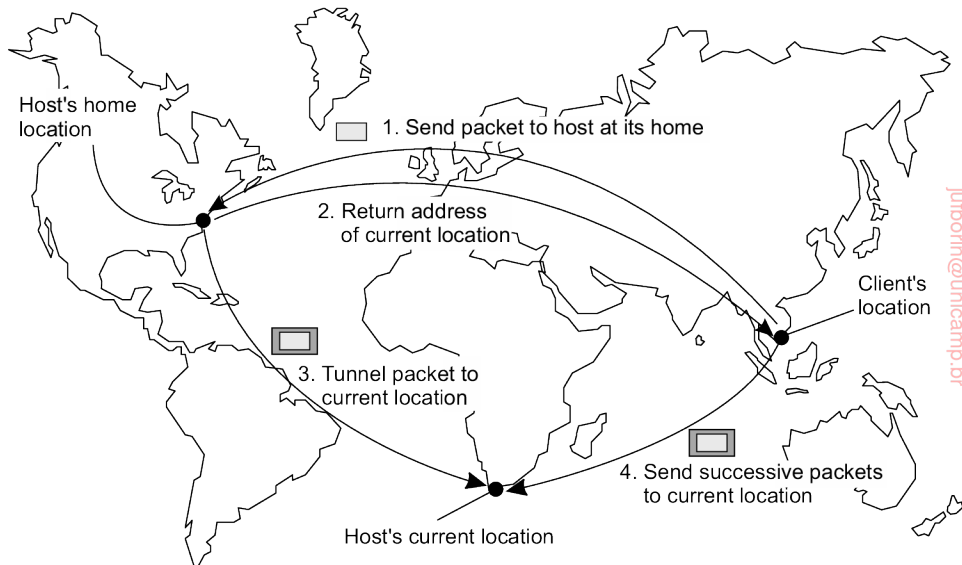


Figure 5.3: The principle of Mobile IP.

contact the home, which may be at a completely different location than the entity itself. The result is an increase in communication latency.

Another drawback of the home-based approach is the use of a fixed home location. For one thing, it must be ensured that the home location always exists. Otherwise, contacting the entity will become impossible. Problems are aggravated when a long-lived entity decides to move permanently to a completely different part of the network than where its home is located. In that case, it would have been better if the home could have moved along with the host.

A solution to this problem is to register the home at a traditional naming service and to let a client first look up the location of the home. Because the home location can be assumed to be relatively stable, that location can be effectively cached after it has been looked up.

Distributed hash tables

Let us now take a closer look at how to resolve an identifier to the address of the associated entity. We have already mentioned distributed hash tables a number of times, but have deferred discussion on how they actually work. In this section we correct this situation by first considering the Chord system as an easy-to-explain DHT-based system.

General mechanism

Many DHT-based systems have been developed in the past decade, with the Chord system [Stoica et al., 2003] being a typical representative. Chord uses an m -bit identifier space to assign randomly chosen identifiers to nodes as well as keys to specific entities. The latter can be virtually anything: files, processes, etc. The number m of bits is usually 128 or 160, depending on which hash function is used. An entity with key k falls under the jurisdiction of the node with the smallest identifier $id \geq k$. This node is referred to as the **successor** of k and denoted as $succ(k)$. To keep our notation simple and consistent, in the following we refer to a node with identifier p as node p .

The main issue in DHT-based systems is to efficiently resolve a key k to the address of $succ(k)$. An obvious nonscalable approach is to let each node p keep track of the successor $succ(p+1)$ as well as its predecessor $pred(p)$. In that case, whenever a node p receives a request to resolve key k , it will simply forward the request to one of its two neighbors—whichever one is appropriate—unless $pred(p) < k \leq p$ in which case node p should return its own address to the process that initiated the resolution of key k .

Instead of this linear approach toward key lookup, each Chord node maintains a **finger table** containing $s \leq m$ entries. If FT_p denotes the finger table of node p , then

$$FT_p[i] = succ(p + 2^{i-1})$$

Put in other words, the i -th entry points to the first node succeeding p by at least 2^{i-1} . Note that these references are actually shortcuts to existing nodes in the identifier space, where the short-cutted distance from node p increases exponentially as the index in the finger table increases. To look up a key k , node p will then immediately forward the request to node q with index j in p 's finger table where:

$$q = FT_p[j] \leq k < FT_p[j+1]$$

or $q = FT_p[1]$ when $p < k < FT_p[1]$. (For clarity, we ignore modulo arithmetic.) Note that when the finger-table size s is equal to 1, a Chord lookup corresponds to naively traversing the ring linearly as we just discussed.

To illustrate this lookup, consider resolving $k = 26$ from node 1 as shown in Figure 5.4. First, node 1 will look up $k = 26$ in its finger table to discover that this value is larger than $FT_1[5]$, meaning that the request will be forwarded to node $18 = FT_1[5]$. Node 18, in turn, will select node 20, as $FT_{18}[2] \leq k < FT_{18}[3]$. Finally, the request is forwarded from node 20 to node 21 and from there to node 28, which is responsible for $k = 26$. At that point, the address of node 28 is returned to node 1 and the key has been resolved. For similar reasons, when node 28 is requested to resolve the key $k = 12$, a request will be routed as shown by the dashed line in Figure 5.4. It can be shown that a lookup will generally require $\mathcal{O}(\log(N))$ steps, with N being the number of nodes in the system.

if q 's successor has updated its predecessor, then apparently a new node p had entered the system, with $q < p \leq \text{succ}(q + 1)$, so that q will adjust $\text{FT}_q[1]$ to p . At that point, it will also check whether p has recorded q as its predecessor. If not, another adjustment of $\text{FT}_q[1]$ is needed.

In a similar way, to update a finger table, node q simply needs to find the successor for $k = q + 2^{i-1}$ for each entry i . Again, this can be done by issuing a request to resolve $\text{succ}(k)$. In Chord, such requests are issued regularly by means of a background process.

Likewise, each node q will regularly check whether its predecessor is alive. If the predecessor has failed, the only thing that q can do is record the fact by setting $\text{pred}(q)$ to "unknown." On the other hand, when node q is updating its link to the next known node in the ring, and finds that the predecessor of $\text{succ}(q + 1)$ has been set to "unknown," it will simply notify $\text{succ}(q + 1)$ that it suspects it to be the predecessor. By and large, these simple procedures ensure that a Chord system is generally consistent, only perhaps with exception of a few nodes. The details can be found in [Stoica et al., 2003].

Note 5.4 (Advanced: Chord in Python)

```

1 class ChordNode:
2     def finger(self, i):
3         succ = (self.nodeID + pow(2, i-1)) % self.MAXPROC      # succ(p+2^(i-1))
4         lwbi = self.nodeSet.index(self.nodeID)                 # self in nodeset
5         upbi = (lwbi + 1) % len(self.nodeSet)                   # next neighbor
6         for k in range(len(self.nodeSet)):                      # process segments
7             if self.inbetween(succ, self.nodeSet[lwbi]+1, self.nodeSet[upbi]+1):
8                 return self.nodeSet[upbi]                      # found successor
9             (lwbi, upbi) = (upbi, (upbi+1) % len(self.nodeSet)) # next segment
10
11     def recomputeFingerTable(self):
12         self.FT[0] = self.nodeSet[self.nodeSet.index(self.nodeID)-1] # Pred.
13         self.FT[1:] = [self.finger(i) for i in range(1, self.nBits+1)] # Succ.
14
15     def localSuccNode(self, key):
16         if self.inbetween(key, self.FT[0]+1, self.nodeID+1): # in (FT[0], self]
17             return self.nodeID                                # responsible node
18         elif self.inbetween(key, self.nodeID+1, self.FT[1]): # in (self, FT[1]]
19             return self.FT[1]                                  # succ. responsible
20         for i in range(1, self.nBits+1):                       # rest of FT
21             if self.inbetween(key, self.FT[i], self.FT[i+1] % self.nBits + 1):
22                 return self.FT[i]                              # in [FT[i], FT[i+1])

```

Figure 5.5: The essence of a Chord node expressed in Python.

Coding Chord in Python is remarkably simple. Again, omitting many of the nonessential coding details, the core of the behavior of a Chord node can be described as shown in Figure 5.5. The function $\text{finger}(i)$ computes $\text{succ}(i)$ for the given node. All nodes known to a specific Chord node are collected in

a local set `nodeSet`, which is sorted by node identifier. The node first looks up its own position in this set, and that of its right-hand neighbor. The operation `inbetween(k, l, u)` computes if $k \in [l, u)$, taking modulo arithmetic into account. Computing `inbetween(k, l+1, u+1)` is therefore the same as testing whether $k \in (l, u]$. We thus see that `finger(i)` returns the largest existing node identifier less or equal to i .

Every time a node learns about a new node in the system (or discovers that one has left), it simply adjusts the local `nodeSet` and recomputes its finger table by calling `recomputeFingerTable`. The finger table itself is implemented as a local table `FT`, with `FT[0]` pointing to the node's predecessor. `nBits` indicates the number of bits used for node identifiers and keys.

The core of what a node does during a lookup is encoded in `localSuccNode(k)`. When handed a key k , it will either return itself, its immediate successor `FT[1]`, or go through the finger table to search the entry satisfying $FT[i] \leq k < FT[i+1]$. The code does not show what is done with the returned value (which is a node identifier), but typically in an *iterative scheme*, the referenced node will be contacted to continue looking up k , unless the node had returned itself as the one being responsible for k . In a *recursive scheme*, the node itself will contact the referenced node.

Note 5.5 (Advanced: Exploiting network proximity)

One of the potential problems with systems such as Chord is that requests may be routed erratically across the Internet. For example, assume that node 1 in Figure 5.5 is placed in Amsterdam, The Netherlands; node 18 in San Diego, California; node 20 in Amsterdam again; and node 21 in San Diego. The result of resolving key 26 will then incur three wide-area message transfers which arguably could have been reduced to at most one. To minimize these pathological cases, designing a DHT-based system requires taking the underlying network into account.

Castro et al. [2002a] distinguish three different ways for making a DHT-based system aware of the underlying network. In the case of **topology-based assignment of node identifiers** the idea is to assign identifiers such that two nearby nodes will have identifiers that are also close to each other. It is not difficult to imagine that this approach may impose severe problems in the case of relatively simple systems such as Chord. In the case where node identifiers are sampled from a one-dimensional space, mapping a logical ring to the Internet is far from trivial. Moreover, such a mapping can easily expose correlated failures: nodes on the same enterprise network will have identifiers from a relatively small interval. When that network becomes unreachable, we suddenly have a gap in the otherwise uniform distribution of identifiers.

With **proximity routing**, nodes maintain a list of alternatives to forward a request to. For example, instead of having only a single successor, each node in Chord could equally well keep track of r successors. In fact, this redundancy can be applied for every entry in a finger table. For node p , `FTp[i]` normally points to

the first node in the range $[p + 2^{i-1}, p + 2^i - 1]$. Whenever it needs to look up key k , it tries to prevent “overshooting” by passing the request to a node q with $k < q$ without knowing for sure if there is a node q' with $k \leq q' < q$. For this reason, p passes k to the node known to p with the *largest* identifier smaller or equal to k .

However, there is no reason why p cannot keep track of r nodes in range $[p + 2^{i-1}, p + 2^i - 1]$: each node q in this range can be used to route a lookup request for a key k as long as $q \leq k$. In that case, when choosing to forward a lookup request, a node can pick one of the r successors that is closest to itself while making sure not to “overshoot.” An additional advantage of having multiple successors for every table entry is that node failures need not immediately lead to failures of lookups, as multiple routes can be explored.

Finally, in **proximity neighbor selection** the idea is to optimize routing tables such that the nearest node is selected as neighbor. This selection works only when there are more nodes to choose from. In Chord, this is normally not the case. However, in other protocols such as Pastry [Rowstron and Druschel, 2001], when a node joins it receives information about the current overlay from multiple other nodes. This information is used by the new node to construct a routing table. Obviously, when there are alternative nodes to choose from, proximity neighbor selection will allow the joining node to choose the best one.

Note that it may not be that easy to draw a line between proximity routing and proximity neighbor selection. In fact, when Chord is modified to include r successors for each finger table entry, proximity neighbor selection resorts to identifying the closest r neighbors, which comes very close to proximity routing as we just explained [Dabek et al., 2004b].

Hierarchical approaches

We now discuss a general approach to a hierarchical location scheme, including a number of optimizations. The approach we present is based on the Globe location service [van Steen et al., 1998]. A detailed description can be found in [Ballintijn, 2003]. This is a general-purpose location service that is representative of many hierarchical location services proposed for what are called Personal Communication Systems, of which a general overview can be found in Pitoura and Samaras [2001].

In a hierarchical scheme, a network is divided into a collection of **domains**. There is a single top-level domain that spans the entire network. Each domain can be subdivided into multiple, smaller subdomains. A lowest-level domain, called a **leaf domain**, typically corresponds to a local-area network in a computer network or a cell in a mobile telephone network. The general assumption is that within a smaller domain the average time it takes to transfer a message from one node to another is less than in a large domain.

Each domain D has an associated directory node $dir(D)$ that keeps track of the entities in that domain. This leads to a tree of directory nodes. The directory node of the top-level domain, called the **root (directory) node**,

knows about all entities. This general organization of a network into domains and directory nodes is illustrated in Figure 5.6

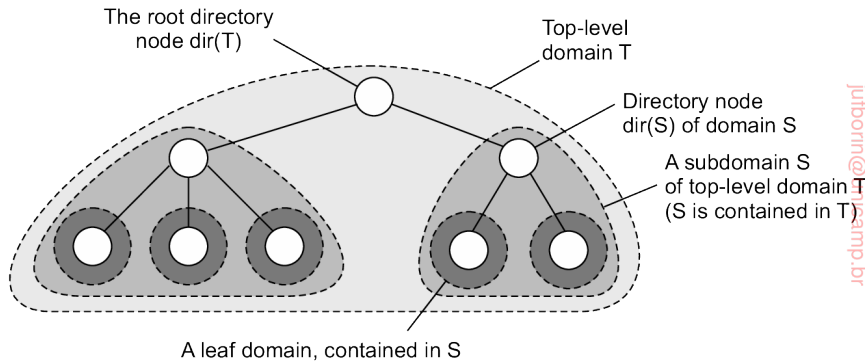


Figure 5.6: Hierarchical organization of a location service into domains, each having an associated directory node.

To keep track of the whereabouts of an entity, each entity currently located in a domain D is represented by a **location record** in the directory node $dir(D)$. A location record for entity E in the directory node N for a leaf domain D contains the entity's current address in that domain. In contrast, the directory node N' for the next higher-level domain D' that contains D , will have a location record for E containing only a pointer to N . Likewise, the parent node of N' will store a location record for E containing only a pointer to N' . Consequently, the root node will have a location record for each entity, where each location record stores a pointer to the directory node of the next lower-level subdomain where that record's associated entity is currently located.

An entity may have multiple addresses, for example if it is replicated. If an entity has an address in leaf domain D_1 and D_2 respectively, then the directory node of the smallest domain containing both D_1 and D_2 , will have two pointers, one for each subdomain containing an address. This leads to the general organization of the tree as shown in Figure 5.7.

Let us now consider how a lookup operation proceeds in such a hierarchical location service. As is shown in Figure 5.8, a client wishing to locate an entity E , issues a lookup request to the directory node of the leaf domain D in which the client resides. If the directory node does not store a location record for the entity, then the entity is currently not located in D . Consequently, the node forwards the request to its parent. Note that the parent node represents a larger domain than its child. If the parent also has no location record for E , the lookup request is forwarded to a next level higher, and so on.

As soon as the request reaches a directory node M that stores a location record for entity E , we know that E is somewhere in the domain $dom(M)$ represented by node M . In Figure 5.8, M is shown to store a location record

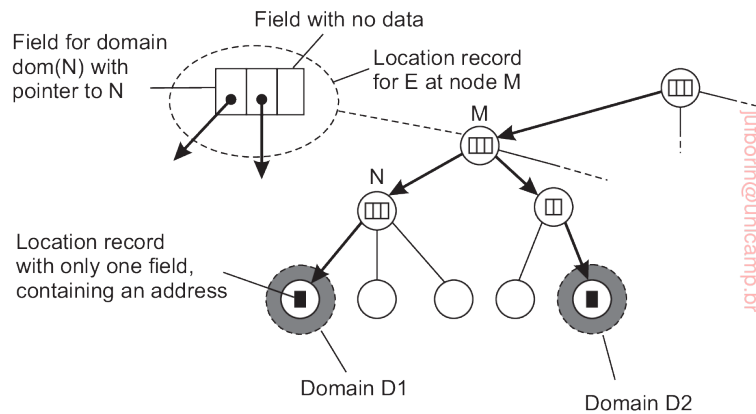


Figure 5.7: An example of storing information of an entity having two addresses in different leaf domains.

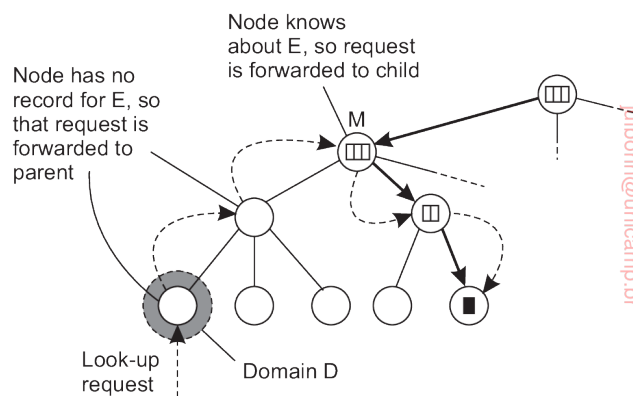


Figure 5.8: Looking up a location in a hierarchically organized location service.

containing a pointer to one of its subdomains. The lookup request is then forwarded to the directory node of that subdomain, which in turn forwards it further down the tree, until the request finally reaches a leaf node. The location record stored in the leaf node will contain the address of E in that leaf domain. This address can then be returned to the client that initially requested the lookup to take place.

An important observation with respect to hierarchical location services is that the lookup operation exploits locality. In principle, the entity is searched in a gradually increasing ring centered around the requesting client. The search area is expanded each time the lookup request is forwarded to a next higher-level directory node. In the worst case, the search continues until the request reaches the root node. Because the root node has a location record for each entity, the request can then simply be forwarded along a downward path of pointers to one of the leaf nodes.

Update operations exploit locality in a similar fashion, as shown in Figure 5.9. Consider an entity E that has created a replica in leaf domain D for which it needs to insert its address. The insertion is initiated at the leaf node $dir(D)$ of D which immediately forwards the insert request to its parent. The parent will forward the insert request as well, until it reaches a directory node M that already stores a location record for E .

Node M will then store a pointer in the location record for E , referring to the child node from where the insert request was forwarded. At that point, the child node creates a location record for E , containing a pointer to the next lower-level node from where the request came. This process continues until we reach the leaf node from which the insert was initiated. The leaf node, finally, creates a record with the entity's address in the associated leaf domain.

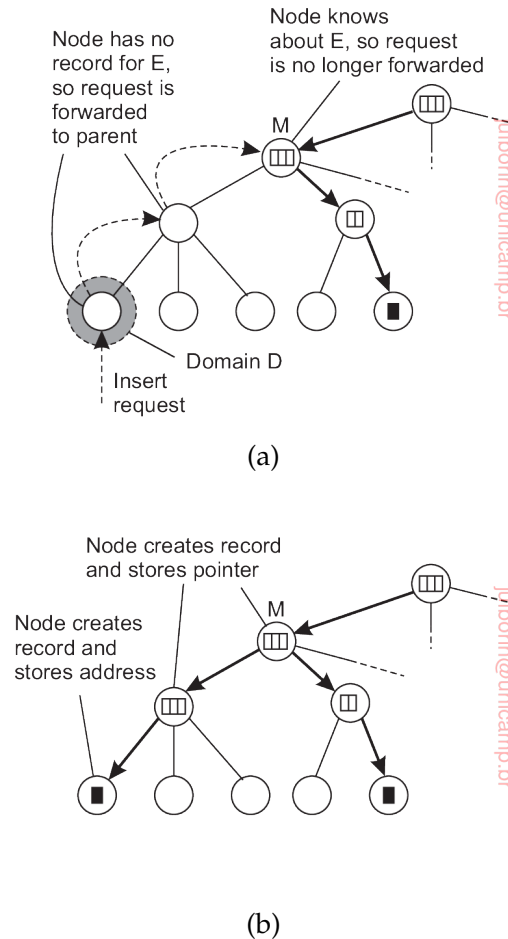


Figure 5.9: (a) An insert request is forwarded to the first node that knows about entity E . (b) A chain of forwarding pointers to the leaf node is created.

Inserting an address as just described leads to installing the chain of pointers in a top-down fashion starting at the lowest-level directory node that has a location record for entity E . An alternative is to create a location record before passing the insert request to the parent node. In other words, the chain of pointers is constructed from the bottom up. The advantage of the latter is that an address becomes available for lookups as soon as possible. Consequently, if a parent node is temporarily unreachable, the address can still be looked up within the domain represented by the current node.

A delete operation is analogous to an insert operation. When an address for entity E in leaf domain D needs to be removed, directory node $dir(D)$ is requested to remove that address from its location record for E . If that location record becomes empty, that is, it contains no other addresses for E in D , the record can be removed. In that case, the parent node of $dir(D)$ wants to remove its pointer to $dir(D)$. If the location record for E at the parent now also becomes empty, that record should be removed as well and the next higher-level directory node should be informed. Again, this process continues until a pointer is removed from a location record that remains nonempty afterward or until the root is reached.

Note 5.6 (Advanced: Scalability issues)

One question that immediately comes to mind is whether the hierarchical approach just described can actually scale. A seemingly obvious design flaw, is that the root node needs to keep track of *all* identifiers. However, it is important to make a distinction between a logical design and its physical implementation. Let us make this distinction here and see how we can actually come to a highly scalable implementation of a hierarchical location service.

To this end, we assume that each entity is assigned a unique identifier uniform at random from a large space of m -bit identifiers, just as in Chord. Furthermore, let us assume that there are a total of N physical hosts $\{H_1, H_2, \dots, H_N\}$ that can accommodate the lookup service, spread across the Internet. Each host is capable of running one or more location servers. Typically, two servers running on the same host will represent two nodes at different levels of the logical tree. Let $D_k(A)$ denote the domain at level k that contains address A , with $k = 0$ denoting the root domain. Likewise, let $LS_k(E, A)$ denote the unique location server in $D_k(A)$ responsible for keeping track of the whereabouts of entity E .

We can now make a distinction between a logical root and its implementation. Let $\mathbf{D}_k = \{D_{k,1}, D_{k,2}, \dots, D_{k,N_k}\}$ denote the N_k domains at level k , with, obviously, $N_0 = |\mathbf{D}_0| = 1$. For each level k , the set of hosts is partitioned into N_k subsets, with each host running a location server representing exactly one of the domains $D_{k,i}$ from $\mathbf{D}_k$. This principle is shown in Figure 5.10.

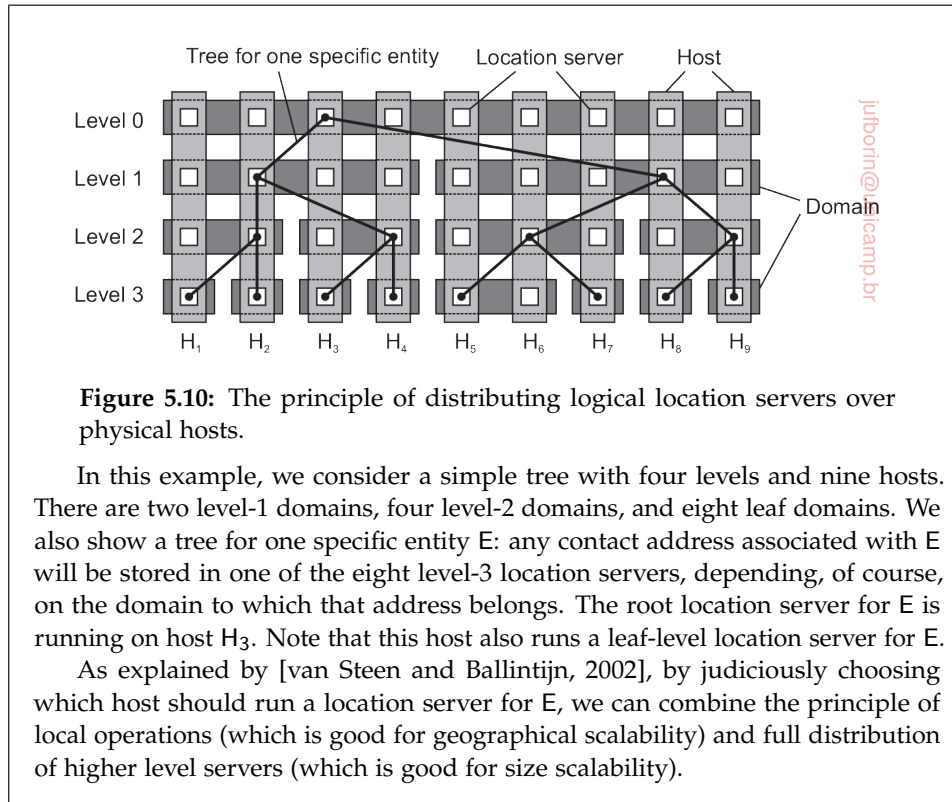


Figure 5.10: The principle of distributing logical location servers over physical hosts.

In this example, we consider a simple tree with four levels and nine hosts. There are two level-1 domains, four level-2 domains, and eight leaf domains. We also show a tree for one specific entity *E*: any contact address associated with *E* will be stored in one of the eight level-3 location servers, depending, of course, on the domain to which that address belongs. The root location server for *E* is running on host *H*₃. Note that this host also runs a leaf-level location server for *E*.

As explained by [van Steen and Ballintijn, 2002], by judiciously choosing which host should run a location server for *E*, we can combine the principle of local operations (which is good for geographical scalability) and full distribution of higher level servers (which is good for size scalability).

5.3 Structured naming

Flat names are good for machines, but are generally not very convenient for humans to use. As an alternative, naming systems generally support structured names that are composed from simple, human-readable names. Not only file naming, but also host naming on the Internet follows this approach. In this section, we concentrate on structured names and the way that these names are resolved to addresses.

Name spaces

Names are commonly organized into what is called a **name space**. Name spaces for structured names can be represented as a labeled, directed graph with two types of nodes. A **leaf node** represents a named entity and has the property that it has no outgoing edges. A leaf node generally stores information on the entity it is representing—for example, its address—so that a client can access it. Alternatively, it can store the state of that entity, such as in the case of file systems in which a leaf node actually contains the complete file it is representing. We return to the contents of nodes below.

In contrast to a leaf node, a **directory node** has a number of outgoing edges, each labeled with a name, as shown in Figure 5.11. Each node in a naming graph is considered as yet another entity in a distributed system, and, in particular, has an associated identifier. A directory node stores a table in which an outgoing edge is represented as a pair (*node identifier, edge label*). Such a table is called a **directory table**.

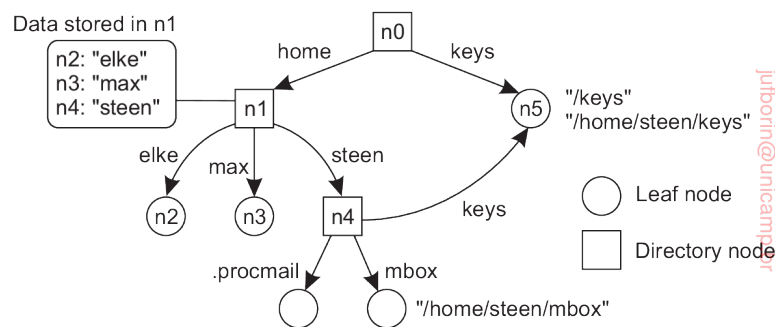


Figure 5.11: A general naming graph with a single root node.

The naming graph shown in Figure 5.11 has one node, namely n_0 , which has only outgoing and no incoming edges. Such a node is called the **root (node)** of the naming graph. Although it is possible for a naming graph to have several root nodes, for simplicity, many naming systems have only one. Each path in a naming graph can be referred to by the sequence of labels corresponding to the edges in that path, such as $N:[label_1, label_2, \dots, label_n]$, where N refers to the first node in the path. Such a sequence is called a **path name**. If the first node in a path name is the root of the naming graph, it is called an **absolute path name**. Otherwise, it is called a **relative path name**.

It is important to realize that names are always organized in a name space. As a consequence, a name is always defined relative only to a directory node. In this sense, the term “absolute name” is somewhat misleading. Likewise, the difference between global and local names can often be confusing. A **global name** is a name that denotes the same entity, no matter where that name is used in a system. In other words, a global name is always interpreted with respect to the same directory node. In contrast, a **local name** is a name whose interpretation depends on where that name is being used. Put differently, a local name is essentially a relative name whose directory in which it is contained is (implicitly) known.

This description of a naming graph comes close to what is implemented in many file systems. However, instead of writing the sequence of edge labels to represent a path name, path names in file systems are generally represented as a single string in which the labels are separated by a special separator character, such as a slash (“/”). This character is also used to indicate whether a path name is absolute. For example, in Figure 5.11 instead

of using $n0:[home, steen, mbox]$, that is, the actual path name, it is common practice to use its string representation `/home/steen/mbox`. Note also that when there are several paths that lead to the same node, that node can be represented by different path names. For example, node $n5$ in Figure 5.11 can be referred to by `/home/steen/keys` as well as `/keys`. The string representation of path names can be equally well applied to naming graphs other than those used for only file systems. In Plan 9 [Pike et al., 1995], all resources, such as processes, hosts, I/O devices, and network interfaces, are named in the same fashion as traditional files. This approach is analogous to implementing a single naming graph for all resources in a distributed system.

There are many different ways to organize a name space. As we mentioned, most name spaces have only a single root node. In many cases, a name space is also strictly hierarchical in the sense that the naming graph is organized as a tree. This means that each node except the root has exactly one incoming edge; the root has no incoming edges. As a consequence, each node also has exactly one associated (absolute) path name.

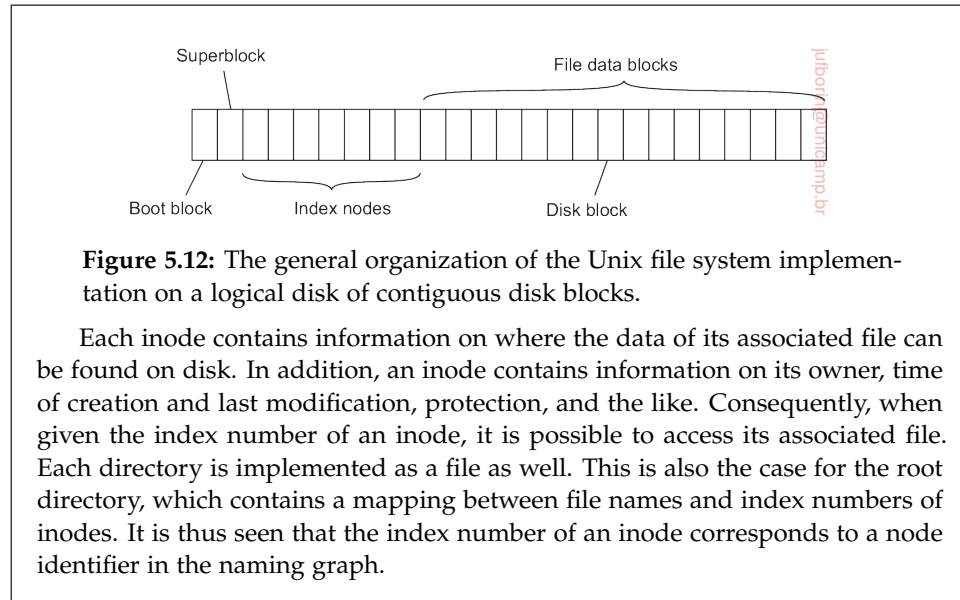
The naming graph shown in Figure 5.11 is an example of **directed acyclic graph**. In such an organization, a node can have more than one incoming edge, but the graph is not permitted to have a cycle. There are also name spaces that do not have this restriction.

Note 5.7 (More information: Implementing the Unix naming graph)

To make matters more concrete, consider the way that files in a traditional Unix file system are named. In a naming graph for Unix a directory node represents a file directory, whereas a leaf node represents a file. There is a single root directory, represented in the naming graph by the root node. The implementation of the naming graph is an integral part of the complete implementation of the file system. That implementation consists of a contiguous series of blocks from a logical disk, generally divided into a boot block, a superblock, a series of index nodes (called inodes), and file data blocks. See also Silberschatz et al. [2012] or Tanenbaum [2001]. This organization is shown in Figure 5.12.

The boot block is a special block of data and instructions that are automatically loaded into main memory when the system is booted. The boot block is used to load the operating system into main memory.

The superblock contains information on the entire file system, such as its size, which blocks on disk are not yet allocated, which inodes are not yet used, and so on. Inodes are referred to by an index number, starting at number zero, which is reserved for the inode representing the root directory.



Name resolution

Name spaces offer a convenient mechanism for storing and retrieving information about entities by means of names. More generally, given a path name, it should be possible to look up any information stored in the node referred to by that name. The process of looking up a name is called **name resolution**.

To explain how name resolution works, let us consider a path name such as $N:[label_1, label_2, \dots, label_n]$. Resolution of this name starts at node N of the naming graph, where the name $label_1$ is looked up in the directory table, and which returns the identifier of the node to which $label_1$ refers. Resolution then continues at the identified node by looking up the name $label_2$ in its directory table, and so on. Assuming that the named path actually exists, resolution stops at the last node referred to by $label_n$, by returning that node's content.

Note 5.8 (More information: The Unix naming graph again)

A name lookup returns the identifier of a node from where the name resolution process continues. In particular, it is necessary to access the directory table of the identified node. Consider again a naming graph for a Unix file system. As mentioned, a node identifier is implemented as the index number of an inode. Accessing a directory table means that first the inode has to be read to find out where the actual data are stored on disk, and then subsequently to read the data blocks containing the directory table.

Closure mechanism

Name resolution can take place only if we know how and where to start. In our example, the starting node was given, and we assumed we had access to its directory table. Knowing how and where to start name resolution is generally referred to as a **closure mechanism**. Essentially, a closure mechanism deals with selecting the initial node in a name space from which name resolution is to start [Radia, 1989]. What makes closure mechanisms sometimes hard to understand is that they are necessarily partly implicit and may be very different when comparing them to each other.

Consider, for example, the string “00312059837784”. Many people will not know what to do with these numbers, unless they are told that the sequence is a telephone number. That information is enough to start the resolution process, in particular, by dialing the number. The telephone system subsequently does the rest.

As another example, consider the use of global and local names in distributed systems. A typical example of a local name is an environment variable. For example, in Unix systems, the variable named HOME is used to refer to the home directory of a user. Each user has its own copy of this variable, which is initialized to the global, systemwide name corresponding to the user’s home directory. The closure mechanism associated with environment variables ensures that the name of the variable is properly resolved by looking it up in a user-specific table.

Note 5.9 (More information: The Unix naming graph and its closure mechanism)

Name resolution in the naming graph for a Unix file system makes use of the fact that the inode of the root directory is the first inode in the logical disk representing the file system. Its actual byte offset is calculated from the values in other fields of the superblock, together with hard-coded information in the operating system itself on the internal organization of the superblock.

To make this point clear, consider the string representation of a file name such as /home/steen/mbox. To resolve this name, it is necessary to already have access to the directory table of the root node of the appropriate naming graph. Being a root node, the node itself cannot have been looked up unless it is implemented as a different node in another naming graph, say *G*. But in that case, it would have been necessary to already have access to the root node of *G*. Consequently, resolving a file name requires that some mechanism has already been implemented by which the resolution process can start.

Linking and mounting

Strongly related to name resolution is the use of **aliases**. An alias is another name for the same entity. An environment variable is an example of an alias. In terms of naming graphs, there are basically two different ways to implement

an alias. The first approach is to simply allow multiple absolute path names to refer to the same node in a naming graph. This approach is illustrated in Figure 5.11, in which node n_5 can be referred to by two different path names. In Unix terminology, both path names `/keys` and `/home/steen/keys` in Figure 5.11 are called **hard links** to node n_5 .

The second approach is to represent an entity by a leaf node, say N , but instead of storing the address or state of that entity, the node stores an absolute path name. When first resolving an absolute path name that leads to N , name resolution will return the path name stored in N , at which point it can continue with resolving that new path name. This principle corresponds to the use of **symbolic links** in Unix file systems, and is illustrated in Figure 5.13. In this example, the path name `/home/steen/keys`, which refers to a node containing the absolute path name `/keys`, is a symbolic link to node n_5 .

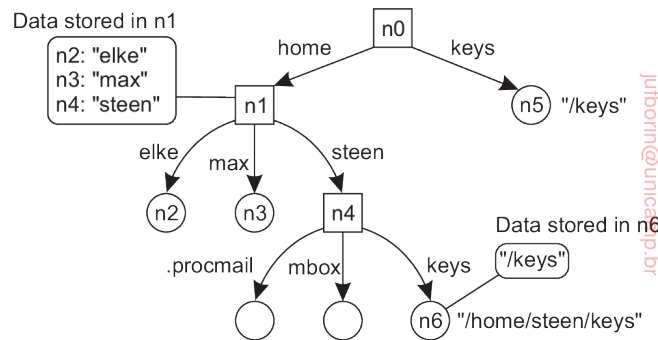


Figure 5.13: The concept of a symbolic link explained in a naming graph.

Name resolution as described so far takes place completely within a single name space. However, name resolution can also be used to merge different name spaces in a transparent way. Let us first consider a mounted file system. In terms of our naming model, a mounted file system corresponds to letting a directory node store the identifier of a directory node from a *different* name space, which we refer to as a **foreign name space**. The directory node storing the node identifier is called a **mount point**. Accordingly, the directory node in the foreign name space is called a **mounting point**. Normally, the mounting point is the root of a name space. During name resolution, the mounting point is looked up and resolution proceeds by accessing its directory table.

The principle of mounting can be generalized to other name spaces as well. In particular, what is needed is a directory node that acts as a mount point and stores all the necessary information for identifying and accessing the mounting point in the foreign name space. This approach is followed in many distributed file systems.

Consider a collection of name spaces that is distributed across different machines. In particular, each name space is implemented by a different server,

each possibly running on a separate machine. Consequently, if we want to mount a foreign name space NS_2 into a name space NS_1 , it may be necessary to communicate over a network with the server of NS_2 , as that server may be running on a different machine than the server for NS_1 . To mount a foreign name space in a distributed system requires at least the following information:

1. The name of an access protocol.
2. The name of the server.
3. The name of the mounting point in the foreign name space.

Note that each of these names needs to be resolved. The name of an access protocol needs to be resolved to the implementation of a protocol by which communication with the server of the foreign name space can take place. The name of the server needs to be resolved to an address where that server can be reached. As the last part in name resolution, the name of the mounting point needs to be resolved to a node identifier in the foreign name space.

In nondistributed systems, none of the three points may actually be needed. For example, in Unix there is no access protocol and no server. Also, the name of the mounting point is not necessary, as it is simply the root directory of the foreign name space.

The name of the mounting point is to be resolved by the server of the foreign name space. However, we also need name spaces and implementations for the access protocol and the server name. One possibility is to represent the three names listed above as a URL.

To make matters concrete, consider a situation in which a user with a laptop computer wants to access files that are stored on a remote file server. The client machine and the file server are both configured with the **Network File System (NFS)**. In particular, to allow NFS to work across the Internet, a client can specify exactly which file it wants to access by means of an NFS URL, for example, `nfs://flits.cs.vu.nl/home/steen`. This URL names a file (which happens to be a directory) called `/home/steen` on an NFS file server `flits.cs.vu.nl`, which can be accessed by a client by means of the NFS protocol [Shepler et al., 2003].

The name `nfs` is a well-known name in the sense that worldwide agreement exists on how to interpret that name. Given that we are dealing with a URL, the name `nfs` will be resolved to an implementation of the NFS protocol. The server name is resolved to its address using DNS, which is discussed in a later section. As we said, `/home/steen` is resolved by the server of the foreign name space.

The organization of a file system on the client machine is partly shown in Figure 5.14. The root directory has a number of user-defined entries, including a subdirectory called `/remote`. This subdirectory is intended to include mount points for foreign name spaces such as the user's home directory at VU

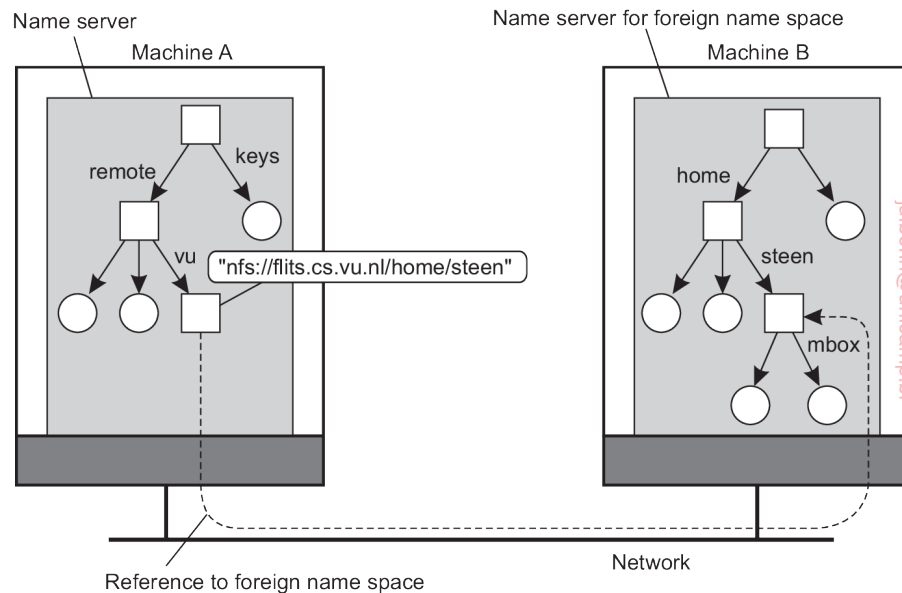


Figure 5.14: Mounting remote name spaces through a specific protocol.

University. To this end, a directory node named `/remote/vu` is used to store the URL `nfs://flits.cs.vu.nl/home/steen`.

Now consider the name `/remote/vu/mbox`. This name is resolved by starting in the root directory on the client's machine and continues until the node `/remote/vu` is reached. The process of name resolution then continues by returning the URL `nfs://flits.cs.vu.nl/home/steen`, in turn leading the client machine to contact the file server `flits.cs.vu.nl` by means of the NFS protocol, and to subsequently access directory `/home/steen`. Name resolution can then be continued by reading the file named `mbox` in that directory, after which the resolution process stops.

Distributed systems that allow mounting a remote file system as just described allow a client machine to, for example, execute the following commands (assume the client machine is named `horton`):

```
horton$ cd /remote/vu
horton$ ls -l
```

which subsequently lists the files in the directory `/home/steen` on the remote file server. The beauty of all this is that the user is spared the details of the actual access to the remote server. Ideally, only some loss in performance is noticed compared to accessing locally available files. In effect, to the client it appears that the name space rooted on the local machine, and the one rooted at `/home/steen` on the remote machine, form a single name space.

Note 5.10 (More information: Mounting across a network in Unix)

There are many ways in which mounting across a network can take place. One practical solution and adopted by many small-scale distributed systems, is to simply assign fixed IP addresses to machines and subsequently offer mounting points to clients. Consider the following example. Suppose we have a Unix machine named *coltrane* using the private address 192.168.2.3, storing a collection of music files under the local directory */audio*. This directory can be *exported* as a mounting point, and as a consequence can be *imported* by another machine.

Let *quandar* be such a machine, and suppose it wants to mount the collection of audio files at the local mount point */home/maarten/Music*. The following command will do the job (assuming the correct privileges have been set):

```
quandar$ mount -t nfs 192.168.2.3:/audio /home/maarten/Music
```

From that moment on, all files available on *coltrane* in its directory */audio* can be accessed by *quandar* in the directory */home/maarten/Music*. The beauty of this scheme is that once mounted, there is no need to think of remote access anymore (until something fails, of course).

The implementation of a name space

A name space forms the heart of a naming service, that is, a service that allows users and processes to add, remove, and look up names. A naming service is implemented by name servers. If a distributed system is restricted to a local-area network, it is often feasible to implement a naming service by means of only a single name server. However, in large-scale distributed systems with many entities, possibly spread across a large geographical area, it is necessary to distribute the implementation of a name space over multiple name servers.

Name space distribution

Name spaces for a large-scale, possibly worldwide distributed system, are usually organized hierarchically. As before, assume such a name space has only a single root node. To effectively implement such a name space, it is convenient to partition it into logical layers. Cheriton and Mann [1989] distinguish the following three layers.

The **global layer** is formed by highest-level nodes, that is, the root node and other directory nodes logically close to the root, namely its children. Nodes in the global layer are often characterized by their stability, in the sense that directory tables are rarely changed. Such nodes may represent organizations, or groups of organizations, for which names are stored in the name space.

The **administrational layer** is formed by directory nodes that together are managed within a single organization. A characteristic feature of the directory

nodes in the administrative layer is that they represent groups of entities that belong to the same organization or administrative unit. For example, there may be a directory node for each department in an organization, or a directory node from which all hosts can be found. Another directory node may be used as the starting point for naming all users, and so forth. The nodes in the administrative layer are relatively stable, although changes generally occur more frequently than to nodes in the global layer.

Finally, the **managerial layer** consists of nodes that may typically change regularly. For example, nodes representing hosts in the local network belong to this layer. For the same reason, the layer includes nodes representing shared files such as those for libraries or binaries. Another important class of nodes includes those that represent user-defined directories and files. In contrast to the global and administrative layer, the nodes in the managerial layer are maintained not only by system administrators, but also by individual end users of a distributed system.

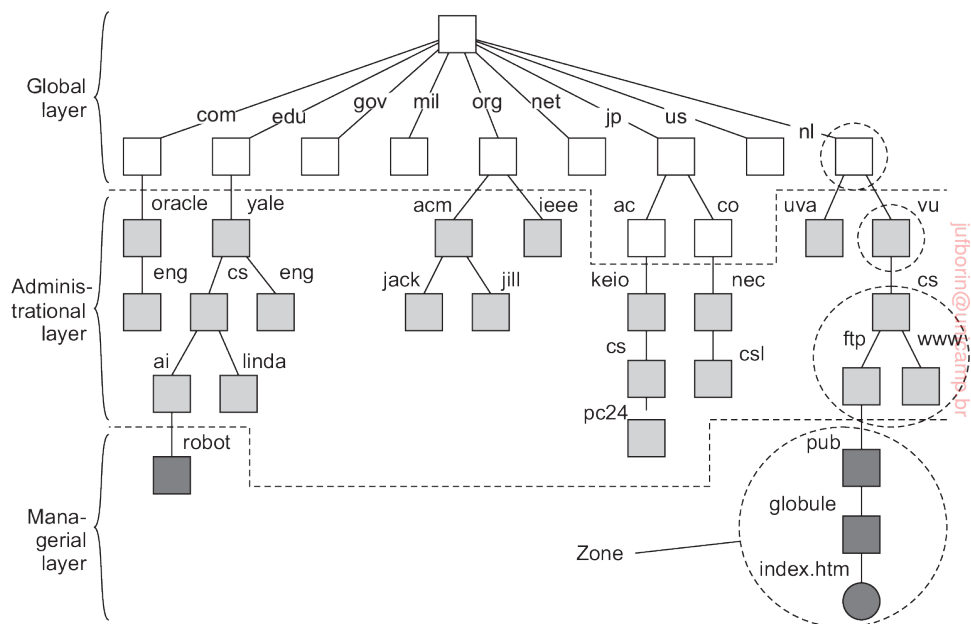


Figure 5.15: An example partitioning of the DNS name space, including Internet-accessible files, into three layers.

To make matters more concrete, Figure 5.15 shows an example of the partitioning of part of the DNS name space, including the names of files within an organization that can be accessed through the Internet, for example, Web pages and transferable files. The name space is divided into nonoverlapping parts, called **zones** in DNS [Mockapetris, 1987]. A zone is a part of the name

space that is implemented by a separate name server. Some of these zones are illustrated in Figure 5.15.

If we take a look at availability and performance, name servers in each layer have to meet different requirements. High availability is especially critical for name servers in the global layer. If a name server fails, a large part of the name space will be unreachable because name resolution cannot proceed beyond the failing server.

Performance is somewhat subtle. Due to the low rate of change of nodes in the global layer, the results of lookup operations generally remain valid for a long time. Consequently, those results can be effectively cached (i.e., stored locally) by the clients. The next time the same lookup operation is performed, the results can be retrieved from the client's cache instead of letting the name server return the results. As a result, name servers in the global layer do not have to respond quickly to a single lookup request. On the other hand, throughput may be important, especially in large-scale systems with millions of users.

The availability and performance requirements for name servers in the global layer can be met by replicating servers, in combination with client-side caching. Updates in this layer generally do not have to come into effect immediately, making it much easier to keep replicas consistent.

Availability for a name server in the administrative layer is primarily important for clients in the same organization as the name server. If the name server fails, many resources within the organization become unreachable because they cannot be looked up. On the other hand, it may be less important that resources in an organization are temporarily unreachable for users outside that organization.

With respect to performance, name servers in the administrative layer have similar characteristics as those in the global layer. Because changes to nodes do not occur all that often, caching lookup results can be highly effective, making performance less critical. However, in contrast to the global layer, the administrative layer should take care that lookup results are returned within a few milliseconds, either directly from the server or from the client's local cache. Likewise, updates should generally be processed quicker than those of the global layer. For example, it is unacceptable that an account for a new user takes hours to become effective.

These requirements can often be met by using relatively powerful machines to run name servers. In addition, client-side caching should be applied, combined with replication for increased overall availability.

Availability requirements for name servers at the managerial level are generally less demanding. In particular, it often suffices to use a single machine to run name servers at the risk of temporary unavailability. However, performance is crucial: operations must take place immediately. Because updates occur regularly, client-side caching is often less effective.

Issue	Global	Administrational	Managerial
Geographical scale	Worldwide	Organization	Department
Number of nodes	Few	Many	Vast numbers
Responsiveness to lookups	Seconds	Milliseconds	Immediate
Update propagation	Lazy	Immediate	Immediate
Number of replicas	Many	None or few	None
Client-side caching	Yes	Yes	Sometimes

Figure 5.16: A comparison between name servers for implementing nodes from a large-scale name space partitioned into a global layer, an administrative layer, and a managerial layer.

A comparison between name servers at different layers is shown in Figure 5.16. In distributed systems, name servers in the global and administrative layer are the most difficult to implement. Difficulties are caused by replication and caching, which are needed for availability and performance, but which also introduce consistency problems. Some of the problems are aggravated by the fact that caches and replicas are spread across a wide-area network, which may introduce long communication delays during lookups.

Implementation of name resolution

The distribution of a name space across multiple name servers affects the implementation of name resolution. To explain the implementation of name resolution in large-scale name services, we assume for the moment that name servers are not replicated and that no client-side caches are used. Each client has access to a local **name resolver**, which is responsible for ensuring that the name resolution process is carried out. Referring to Figure 5.15, assume the (absolute) path name `root:[nl, vu, cs, ftp, pub, globe, index.html]` is to be resolved. Using a URL notation, this path name would correspond to `ftp://ftp.cs.vu.nl/pub/globe/index.html`. There are now two ways to implement name resolution.

In **iterative name resolution**, a name resolver hands over the complete name to the root name server. It is assumed that the address where the root server can be contacted is well known. The root server will resolve the path name as far as it can, and return the result to the client. In our example, the root server can resolve only the label `nl`, for which it will return the address of the associated name server.

At that point, the client passes the remaining path name (i.e., `nl:[vu, cs, ftp, pub, globe, index.html]`) to that name server. This server can resolve only the label `vu`, and returns the address of the associated name server, along with the remaining path name `vu:[cs, ftp, pub, globe, index.html]`.

The client's name resolver will then contact this next name server, which responds by resolving the label `cs`, and subsequently also `ftp`, returning the

address of the FTP server along with the path name ftp:[pub,globe,index.html]. The client then contacts the FTP server, requesting it to resolve the last part of the original path name. The FTP server will subsequently resolve the labels pub, globe, and index.html, and transfer the requested file (in this case using FTP). This process of iterative name resolution is shown in Figure 5.17. (The notation #[cs] is used to indicate the address of the server responsible for handling the node referred to by [cs].)

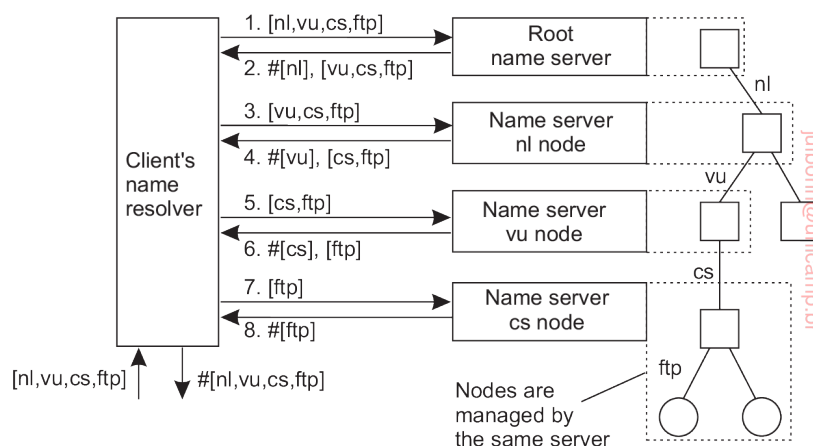


Figure 5.17: The principle of iterative name resolution.

In practice, the last step, namely contacting the FTP server and requesting it to transfer the file with path name ftp:[pub,globe,index.html], is carried out separately by the client process. In other words, the client would normally hand only the path name root:[nl,vu,cs,ftp] to the name resolver, from which it would expect the address where it can contact the FTP server, as is also shown in Figure 5.17

An alternative to iterative name resolution is to use recursion during name resolution. Instead of returning each intermediate result back to the client's name resolver, with recursive name resolution, a name server passes the result to the next name server it finds. So, for example, when the root name server finds the address of the name server implementing the node named nl, it requests that name server to resolve the path name nl:[vu,cs,ftp, pub,globe,index.html]. Using recursive name resolution as well, this next server will resolve the complete path and eventually return the file index.html. to the root server, which, in turn, will pass that file to the client's name resolver.

Recursive name resolution is shown in Figure 5.18. As in iterative name resolution, the last step (contacting the FTP server and asking it to transfer the indicated file) is generally carried out as a separate process by the client.

The main drawback of recursive name resolution is that it puts a higher

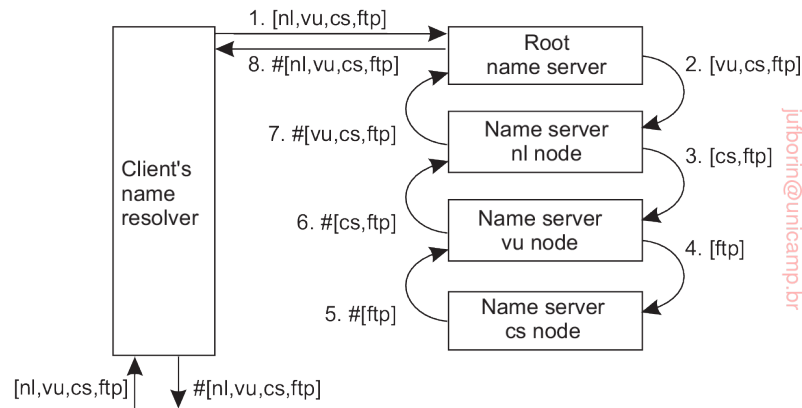


Figure 5.18: The principle of recursive name resolution.

performance demand on each name server. Basically, a name server is required to handle the complete resolution of a path name, although it may do so in cooperation with other name servers. This additional burden is generally so high that name servers in the global layer of a name space support only iterative name resolution.

There are two important advantages to recursive name resolution. The first advantage is that caching results is more effective compared to iterative name resolution. The second advantage is that communication costs may be reduced. To explain these advantages, assume that a client's name resolver will accept path names referring only to nodes in the global or administrative layer of the name space. To resolve that part of a path name that corresponds to nodes in the managerial layer, a client will separately contact the name server returned by its name resolver, as we discussed above.

Recursive name resolution allows each name server to gradually learn the address of each name server responsible for implementing lower-level nodes. As a result, caching can be effectively used to enhance performance. For example, when the root server is requested to resolve the path name `root:[nl,vu,cs,ftp]`, it will eventually get the address of the name server implementing the node referred to by that path name. To come to that point, the name server for the `nl` node has to look up the address of the name server for the `vu` node, whereas the latter has to look up the address of the name server handling the `cs` node.

Because changes to nodes in the global and administrative layer do not occur often, the root name server can effectively cache the returned address. Moreover, because the address is also returned, by recursion, to the name server responsible for implementing the `vu` node and to the one implementing the `nl` node, it might as well be cached at those servers too.

Likewise, the results of intermediate name lookups can also be returned

and cached. For example, the server for the nl node will have to look up the address of the vu node server. That address can be returned to the root server when the nl server returns the result of the original name lookup. A complete overview of the resolution process, and the results that can be cached by each name server is shown in Figure 5.19.

Server for node	Should resolve	Looks up	Passes to child	Receives and caches	Returns to requester
cs	[ftp]	#[ftp]	—	—	#[ftp]
vu	[cs, ftp]	#[cs]	[ftp]	#[ftp]	#[cs] #[cs, ftp]
nl	[vu, cs, ftp]	#[vu]	[cs, ftp]	#[cs] #[cs, ftp]	#[vu] #[vu, cs] #[vu, cs, ftp]
root	[nl, vu, cs, ftp]	#[nl]	[vu, cs, ftp]	#[vu] #[vu, cs] #[vu, cs, ftp]	#[nl] #[nl, vu] #[nl, vu, cs] #[nl, vu, cs, ftp]

Figure 5.19: Recursive name resolution of [nl, vu, cs, ftp]. Name servers cache intermediate results for subsequent lookups.

The main benefit of this approach is that, eventually, lookup operations can be handled quite efficiently. For example, suppose that another client later requests resolution of the path name root:[nl, vu, cs, flits]. This name is passed to the root, which can immediately forward it to the name server for the cs node, and request it to resolve the remaining path name cs:[flits].

With iterative name resolution, caching is necessarily restricted to the client's name resolver. Consequently, if a client A requests the resolution of a name, and another client B later requests that same name to be resolved, name resolution will have to pass through the same name servers as was done for client A. As a compromise, many organizations use a local, intermediate name server that is shared by all clients. This local name server handles all naming requests and caches results. Such an intermediate server is also convenient from a management point of view. For example, only that server needs to know where the root name server is located; other machines do not require this information.

The second advantage of recursive name resolution is that it is often cheaper with respect to communication. Again, consider the resolution of the path name root:[nl, vu, cs, ftp] and assume the client is located in San Francisco. Assuming that the client knows the address of the server for the nl node, with recursive name resolution, communication follows the route from the client's host in San Francisco to the nl server in The Netherlands, shown as R1 in Figure 5.20. From there on, communication is subsequently needed

between the nl server and the name server of VU University on the campus in Amsterdam, The Netherlands. This communication is shown as R2. Finally, communication is needed between the vu server and the name server in the Computer Science Department, shown as R3. The route for the reply is the same, but in the opposite direction. Clearly, communication costs are dictated by the message exchange between the client's host and the nl server.

In contrast, with iterative name resolution, the client's host has to communicate separately with the nl server, the vu server, and the cs server, of which the total costs may be roughly three times that of recursive name resolution. The arrows in Figure 5.20 labeled I1, I2, and I3 show the communication path for iterative name resolution.

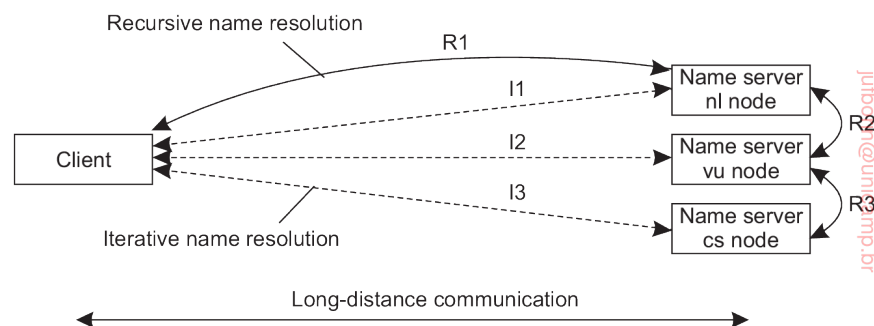


Figure 5.20: The comparison between recursive and iterative name resolution with respect to communication costs.

Example: The Domain Name System

One of the largest distributed naming services in use today is the Internet **Domain Name System (DNS)**. DNS is primarily used for looking up IP addresses of hosts and mail servers. In the following pages, we concentrate on the organization of the DNS name space and the information stored in its nodes. Also, we take a closer look at the actual implementation of DNS. More information can be found in [Mockapetris, 1987] and [Liu and Albitz, 2006]. An assessment of DNS, notably concerning whether it still fits the needs of the current Internet, can be found in [Levien, 2005]. From this report, one can draw the somewhat surprising conclusion that even after more than 30 years, DNS gives no indication that it needs to be replaced. We would argue that the main cause lies in the designer's deep understanding of how to keep matters simple. Practice in other fields of distributed systems indicates that not many are gifted with such an understanding.

The DNS name space

The DNS name space is hierarchically organized as a rooted tree. A label is a case-insensitive string made up of alphanumeric characters. A label has a maximum length of 63 characters; the length of a complete path name is restricted to 255 characters. The string representation of a path name consists of listing its labels, starting with the rightmost one, and separating the labels by a dot ("."). The root is represented by a dot. So, for example, the path name root:[nl,vu,cs,flits], is represented by the string "flits.cs.vu.nl.", which includes the rightmost dot to indicate the root node. We generally omit this dot for readability.

Because each node in the DNS name space has exactly one incoming edge (with the exception of the root node, which has no incoming edges), the label attached to a node's incoming edge is also used as the name for that node. A subtree is called a **domain**; a path name to its root node is called a **domain name**. Note that, just like a path name, a domain name can be either absolute or relative.

The contents of a node is formed by a collection of **resource records**. There are different types of resource records. The major ones are shown in Figure 5.21.

Type	Refers to	Description
SOA	Zone	Holds info on the represented zone
A	Host	IP addr. of host this node represents
MX	Domain	Mail server to handle mail for this node
SRV	Domain	Server handling a specific service
NS	Zone	Name server for the represented zone
CNAME	Node	Symbolic link
PTR	Host	Canonical name of a host
HINFO	Host	Info on this host
TXT	Any kind	Any info considered useful

Figure 5.21: The most important types of resource records forming the contents of nodes in the DNS name space.

A node in the DNS name space will often represent several entities at the same time. For example, a domain name such as vu.nl is used to represent a domain and a zone. In this case, the domain is implemented by means of several (nonoverlapping) zones.

An SOA (start of authority) resource record contains information such as an e-mail address of the system administrator responsible for the represented zone, the name of the host where data on the zone can be fetched, and so on.

An A (address) record, represents a particular host in the Internet. The A record contains an IP address for that host to allow communication. If a host has several IP addresses, as is the case with multi-homed machines, the node will contain an A record for each address.

Another type of record is the MX (mail exchange) record, which is like a symbolic link to a node representing a mail server. For example, the node representing the domain `cs.vu.nl` has an MX record containing the name `zephyr.cs.vu.nl` which refers to a mail server. That server will handle all incoming mail addressed to users in the `cs.vu.nl` domain. There may be several MX records stored in a node.

Related to MX records are SRV records, which contain the name of a server for a specific service. The service itself is identified by means of a name along with the name of a protocol. For example, the Web server in the `cs.vu.nl` domain could be named by means of an SRV record such as `_http_tcp.cs.vu.nl`. This record would then refer to the actual name of the server (which is `soling.cs.vu.nl`). An important advantage of SRV records is that clients need no longer know the DNS name of the host providing a specific service. Instead, only service names need to be standardized, after which the providing host can be looked up.

Nodes that represent a zone, contain one or more NS (name server) records. Like MX records, an NS record contains the name of a name server that implements the zone represented by the node. In principle, each node in the name space can store an NS record referring to the name server that implements it. However, as we discuss below, the implementation of the DNS name space is such that only nodes representing zones need to store NS records.

DNS distinguishes aliases from what are called **canonical names**. Each host is assumed to have a canonical, or primary name. An alias is implemented by means of node storing a CNAME record containing the canonical name of a host. The name of the node storing such a record is thus the same as a symbolic link, as was shown in Figure 5.13.

DNS maintains an inverse mapping of IP addresses to host names by means of PTR (pointer) records. To accommodate the lookups of host names when given only an IP address, DNS maintains a domain named `in-addr.arpa`, which contains nodes that represent Internet hosts and which are named by the IP address of the represented host. For example, host `www.cs.vu.nl` has IP address `130.37.20.20`. DNS creates a node named `20.20.37.130.in-addr.arpa`, which is used to store the canonical name of that host (which happens to be `soling.cs.vu.nl` in a PTR record).

Finally, an HINFO (host info) record is used to store additional information on a host such as its machine type and operating system. In a similar fashion, TXT records are used for any other kind of data that a user finds useful to store about the entity represented by the node.

DNS implementation

In essence, the DNS name space can be divided into a global layer and an administrative layer as shown in Figure 5.15. The administrative layer, which is generally formed by local file systems, is formally not part of DNS and is therefore also not managed by it.

Each zone is implemented by a name server, which is virtually always replicated for availability. Updates for a zone are normally handled by the primary name server. Updates take place by modifying the DNS database local to the primary server. Secondary name servers do not access the database directly, but, instead, request the primary server to transfer its content. The latter is called a **zone transfer** in DNS terminology.

A DNS database is implemented as a (small) collection of files, of which the most important one contains all the resource records for *all* the nodes in a particular zone. This approach allows nodes to be simply identified by means of their domain name, by which the notion of a node identifier reduces to an (implicit) index into a file.

Note 5.11 (More information: An example DNS database)

To better understand these implementation issues, Figure 5.22 shows a small part of the file that contains most of the information for a previous organization of the cs.vu.nl domain. Note that we have deliberately chosen an outdated version for security reasons. The file has been edited for readability. It shows the content of several nodes that are part of the cs.vu.nl domain, where each node is identified by means of its domain name.

The node cs.vu.nl represents the domain as well as the zone. Its SOA resource record contains specific information on the validity of this file, which will not concern us further. There are four name servers for this zone, referred to by their canonical host names in the NS records. The TXT record is used to give some additional information on this zone, but cannot be automatically processed by any name server. Furthermore, there is a single mail server that can handle incoming mail addressed to users in this domain. The number preceding the name of a mail server specifies a selection priority. A sending mail server should always first attempt to contact the mail server with the lowest number.

The host star.cs.vu.nl operates as a name server for this zone. Name servers are critical to any naming service. What can be seen about this name server is that additional robustness has been created by giving two separate network interfaces, each represented by a separate A resource record. In this way, the effects of a broken network link can be somewhat alleviated as the server will remain accessible.

The next four lines (for zephyr.cs.vu.nl) give the necessary information about one of the department's mail servers. Note that this mail server is also backed up by another mail server, whose path is tornado.cs.vu.nl.

Name	Record type	Record value
cs.vu.nl.	SOA	star.cs.vu.nl. hostmaster.cs.vu.nl. 2005092900 7200 3600 2419200 3600
cs.vu.nl.	TXT	"VU University - Computer Science"
cs.vu.nl.	MX	1 mail.few.vu.nl.
cs.vu.nl.	NS	ns.vu.nl.
cs.vu.nl.	NS	top.cs.vu.nl.
cs.vu.nl.	NS	solo.cs.vu.nl.
cs.vu.nl.	NS	star.cs.vu.nl.
star.cs.vu.nl.	A	130.37.24.6
star.cs.vu.nl.	A	192.31.231.42
star.cs.vu.nl.	MX	1 star.cs.vu.nl.
star.cs.vu.nl.	MX	666 zephyr.cs.vu.nl.
star.cs.vu.nl.	HINFO	"Sun" "Unix"
zephyr.cs.vu.nl.	A	130.37.20.10
zephyr.cs.vu.nl.	MX	1 zephyr.cs.vu.nl.
zephyr.cs.vu.nl.	MX	2 tornado.cs.vu.nl.
zephyr.cs.vu.nl.	HINFO	"Sun" "Unix"
ftp.cs.vu.nl.	CNAME	soling.cs.vu.nl.
www.cs.vu.nl.	CNAME	soling.cs.vu.nl.
soling.cs.vu.nl.	A	130.37.20.20
soling.cs.vu.nl.	MX	1 soling.cs.vu.nl.
soling.cs.vu.nl.	MX	666 zephyr.cs.vu.nl.
soling.cs.vu.nl.	HINFO	"Sun" "Unix"
vucs-das1.cs.vu.nl.	PTR	0.198.37.130.in-addr.arpa.
vucs-das1.cs.vu.nl.	A	130.37.198.0
inkt.cs.vu.nl.	HINFO	"OCE" "Proprietary"
inkt.cs.vu.nl.	A	192.168.4.3
pen.cs.vu.nl.	HINFO	"OCE" "Proprietary"
pen.cs.vu.nl.	A	192.168.4.2
localhost.cs.vu.nl.	A	127.0.0.1

Figure 5.22: An excerpt from an (old) DNS database for the zone cs.vu.nl.

The next six lines show a typical configuration in which the department's Web server, as well as the department's FTP server are implemented by a single machine, called soling.cs.vu.nl. By executing both servers on the same machine (and essentially using that machine only for Internet services and not anything else), system management becomes easier. For example, both servers will have the same view of the file system, and for efficiency, part of the file system may be implemented on soling.cs.vu.nl. This approach is often applied in the case of WWW and FTP services.

The following two lines show information on one of the department's older server clusters. In this case, it tells us that the address 130.37.198.0 is associated with the host name vucs-das1.cs.vu.nl.

The next four lines show information on two major printers connected to the

local network. Note that addresses in the range 192.168.0.0 to 192.168.255.255 are private: they can be accessed only from inside the local network and are not accessible from an arbitrary Internet host.

Name	Record type	Record value
cs.vu.nl.	NS	solo.cs.vu.nl.
cs.vu.nl.	NS	star.cs.vu.nl.
cs.vu.nl.	NS	ns.vu.nl.
cs.vu.nl.	NS	top.cs.vu.nl.
ns.vu.nl.	A	130.37.129.4
top.cs.vu.nl.	A	130.37.20.4
solo.cs.vu.nl.	A	130.37.20.5
star.cs.vu.nl.	A	130.37.24.6
star.cs.vu.nl.	A	192.31.231.42

Figure 5.23: Part of the description for the vu.nl domain which contains the cs.vu.nl domain.

Because the cs.vu.nl domain is implemented as a single zone, Figure 5.22 does not include references to other zones. The way to refer to nodes in a subdomain that are implemented in a different zone is shown in Figure 5.23. What needs to be done is to specify a name server for the subdomain by simply giving its domain name and IP address. When resolving a name for a node that lies in the cs.vu.nl domain, name resolution will continue at a certain point by reading the DNS database stored by the name server for the cs.vu.nl domain.

Note 5.12 (Advanced: Decentralized versus hierarchical DNS implementations)

The implementation of DNS we described so far is the standard one. It follows a hierarchy of servers with 13 well-known root nodes and ending in millions of servers at the leaves (but read on). An important observation is that higher-level nodes receive many more requests than lower-level nodes. Only by caching the name-to-address bindings of these higher levels, it becomes possible to avoid sending requests to them and thus swamping them.

These scalability problems can, in principle, be avoided altogether with fully decentralized solutions. In particular, we can compute the hash of a DNS name, and subsequently take that hash as a key value to be looked up in a distributed hash table or a hierarchical location service with a fully partitioned root node. The obvious drawback of this approach is that we lose the structure of the original name. This loss may prevent efficient implementations of, for example, finding all children in a specific domain.

As argued by Walfish et al. [2004], when there is a need for many names, using identifiers as a semantic-free way of accessing data will allow different systems to make use of a single naming system. The reason is simple: by now it is well understood how a huge collection of (flat) names can be efficiently supported. What needs to be done is to maintain the mapping of identifier-to-

name information, where in this case a name may come from the DNS space, be a URL, and so on. Using identifiers can be made easier by letting users or organizations use a strict local name space. The latter is completely analogous to maintaining a private setting of environment variables on a computer.

Nevertheless, stating that a decentralized implementation of DNS will circumvent many of its scalability problems is too simple. In a comparative study, Pappas et al. [2006] showed that there are many trade-offs to consider and that actually the current, hierarchical design of DNS is not so bad for at least two reasons:

- In a hierarchical design, not all nodes are equal and in the case of DNS, notably the higher-level nodes are *engineered* differently. For example, despite that there are officially 13 root nodes, each of these nodes is highly distributed and replicated for performance and availability. To illustrate, the root node provided by RIPE NCC is implemented at some 25 different sites (all using the same IP address), each implemented as a highly robust and replicated server cluster.

Again, we see the important difference between a logical and physical design. Exploiting this difference is crucial for the operation of a distributed system such as DNS. However, in virtually all DHT-based systems, making this distinction can be much more difficult when dealing with a logical naming hierarchy, as all names are necessarily treated to be equal. In such cases, it becomes much more difficult to engineer the system so that, for example, top-level domains are separated out by special (physical) nodes. Of course, the obvious drawback of not having all nodes being equal, is that special measures need to be taken to protect the more important parts of a system against abuse. We have already mentioned that top-level nodes in DNS are implemented as distributed and replicated server (clusters), but also that an associated server will not provide recursive name resolution. Such implementation decisions are necessary also from a perspective of robustness.

- DNS caches are highly effective and driven almost entirely by the local distribution of queries: if domain D is queried often at server S, then the references for name servers of D will be cached at S. The behavior at another server S' is determined by what is queried at S'. This important feature has been confirmed in a more recent study that also shows how difficult it can be to understand the effectiveness of caching and the locality principles of DNS resolvers. In particular, an ISP's DNS resolver may be very effective in redirecting traffic to content that is localized in that ISP [Ager et al., 2010]. In contrast, caching and replication in DHT-based systems generally does not show such principles of locality: results are simply cached at nodes on the return path of a lookup and have very little to do with the fact that a lookup was locally initiated at a specific node in the DHT, or a resolver for which the local ISP can assist in looking up content.

The fact remains that replacing DNS by a decentralized implementation is not necessarily a good idea. DNS as it stands today, is a well-engineered system that

is difficult to beat when it comes to performance and robustness [Vixie, 2009; 2014].

Example: The Network File System

As another, and very different example, consider naming in NFS. The fundamental idea underlying the NFS naming model is to provide clients complete transparent access to a remote file system as maintained by a server. This transparency is achieved by letting a client be able to mount a remote file system into its own local file system, as shown in Figure 5.24.

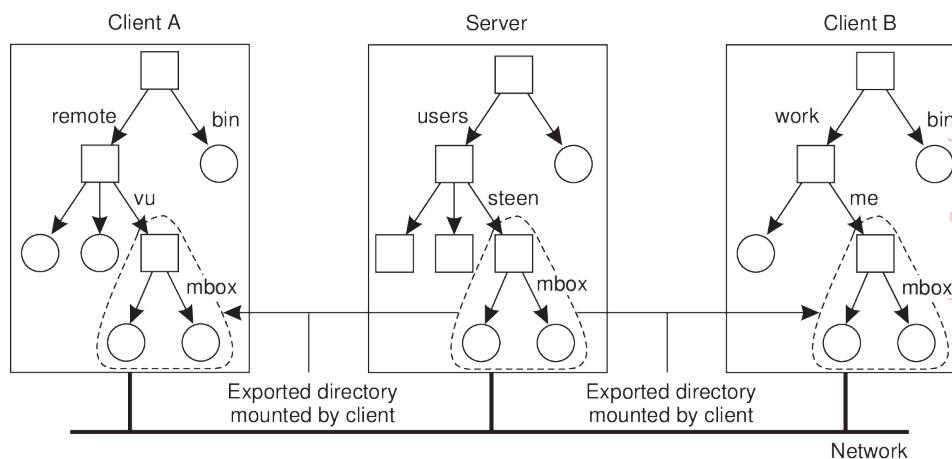


Figure 5.24: Mounting (part of) a remote file system in NFS.

Instead of mounting an entire file system, NFS allows clients to mount only part of a file system, as also shown in Figure 5.24. A server is said to **export** a directory when it makes that directory and its entries available to clients. An exported directory can be mounted into a client's local name space.

This design approach has a serious implication: in principle, users do not share name spaces. As shown in Figure 5.24 the file named `/remote/vu/mbox` at client A is named `/work/me/mbox` at client B. A file's name therefore depends on how clients organize their own local name space, and where exported directories are mounted. The drawback of this approach in a distributed file system is that sharing files becomes much harder. For example, Alice cannot tell Bob about a file using the name she assigned to that file, for that name may have a completely different meaning in Bob's name space of files.

There are several ways to solve this problem, but the most common one is to provide each client with a name space that is partly standardized. For

example, each client may be using the local directory `/usr/bin` to mount a file system containing a standard collection of programs that are available to everyone. Likewise, the directory `/local` may be used as a standard to mount a local file system that is located on the client's host.

An NFS server can itself mount directories that are exported by other servers. However, it is not allowed to export those directories to its own clients. Instead, a client will have to explicitly mount such a directory from the server that maintains it, as shown in Figure 5.25. This restriction comes partly from simplicity. If a server could export a directory that it mounted from another server, it would have to return special file handles that include an identifier for a server. NFS does not support such file handles.

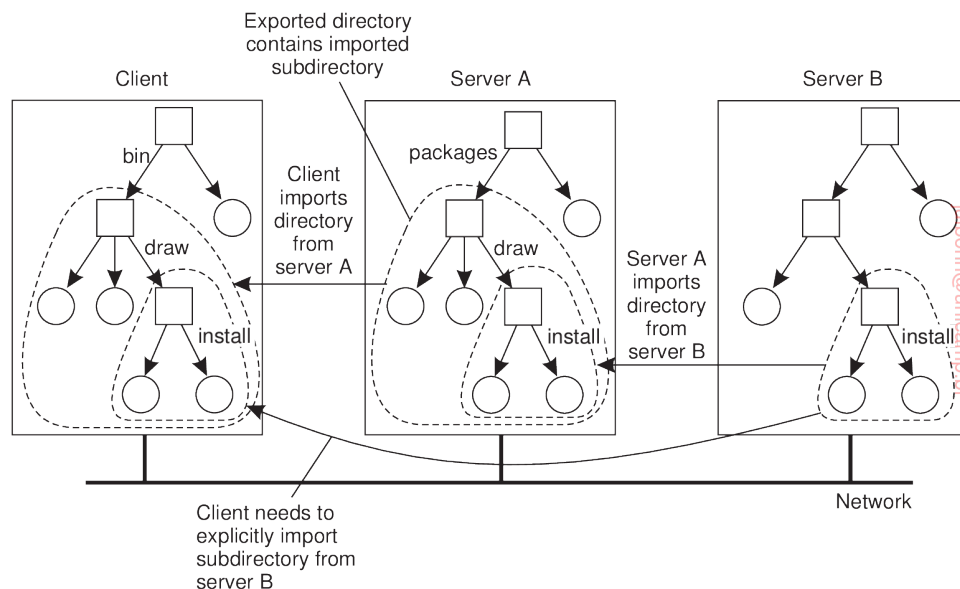


Figure 5.25: Mounting nested directories from multiple servers in NFS.

To explain this point in more detail, assume that server A hosts a file system FS_A from which it exports the directory `/packages`. This directory contains a subdirectory `/draw` that acts as a mount point for a file system FS_B that is exported by server B and mounted by A. Let A also export `/packages/draw` to its own clients, and assume that a client has mounted `/packages` into its local directory `/bin` as shown in Figure 5.25.

If name resolution is iterative, then to resolve the name `/bin/draw/install`, the client contacts server A when it has locally resolved `/bin` and requests A to return a file handle for directory `/draw`. In that case, server A should return a file handle that includes an identifier for server B, for only B can resolve the rest of the path name, in this case `/install`. As we have said, this kind of name resolution is not supported by NFS.

Name resolution in earlier versions of NFS is strictly iterative in the sense that only a single file name at a time can be looked up. In other words, resolving a name such as `/bin/draw/install` requires three separate calls to the NFS server. Moreover, the client is fully responsible for implementing the resolution of a path name. NFSv4 also supports recursive name lookups. In this case, a client can pass a complete path name to a server and request that server to resolve it.

There is another peculiarity with NFS name lookups that has been solved with the most recent version (NFSv4). Consider a file server hosting several file systems. With the strict iterative name resolution, whenever a lookup is done for a directory on which another file system was mounted, the lookup would return the file handle of the directory. Subsequently reading that directory would return its *original* content, not that of the root directory of the mounted file system.

To explain, assume that in our previous example that both file systems FS_A and FS_B are hosted by a single server. If the client has mounted `/packages` into its local directory `/bin`, then looking up the file name `draw` at the server would return the file handle for `draw`. A subsequent call to the server for listing the directory entries of `draw` by means of `readdir` would then return the list of directory entries that were originally stored in FS_A in subdirectory `/packages/draw`. Only if the client had also mounted file system FS_B , would it be possible to properly resolve the path name `draw/install` relative to `/bin`.

NFSv4 solves this problem by allowing lookups to cross mount points at a server. In particular, lookup returns the file handle of the *mounted* directory instead of that of the original directory. The client can detect that the lookup has crossed a mount point by inspecting the file system identifier of the looked up file. If required, the client can locally mount that file system as well.

A file handle is a reference to a file within a file system. It is independent of the name of the file it refers to. A file handle is created by the server that is hosting the file system and is unique with respect to all file systems exported by the server. It is created when the file is created. The client is kept ignorant of the actual content of a file handle; it is completely opaque. File handles were 32 bytes in NFS version 2, but were variable up to 64 bytes in version 3 and 128 bytes in version 4. Of course, the length of a file handle is not opaque.

Ideally, a file handle is implemented as a true identifier for a file relative to a file system. For one thing, this means that as long as the file exists, it should have one and the same file handle. This persistence requirement allows a client to store a file handle locally once the associated file has been looked up by means of its name. One benefit is performance: as most file operations require a file handle instead of a name, the client can avoid having to look up a name repeatedly before every file operation. Another benefit of this approach is that the client can now access the file regardless which (current) name it has.

Because a file handle can be locally stored by a client, it is also important that a server does not reuse a file handle after deleting a file. Otherwise, a client may mistakenly access the wrong file when it uses its locally stored file handle.

Note that the combination of iterative name lookups and not letting a lookup operation allow crossing a mount point introduces a problem with getting an initial file handle. In order to access files in a remote file system, a client will need to provide the server with a file handle of the directory where the lookup should take place, along with the name of the file or directory that is to be resolved. NFSv3 solves this problem through a separate mount protocol, by which a client actually mounts a remote file system. After mounting, the client is passed back the **root file handle** of the mounted file system, which it can subsequently use as a starting point for looking up names.

In NFSv4, this problem is solved by providing a separate operation `putrootfh` that tells the server to solve all file names relative to the root file handle of the file system it manages. The root file handle can be used to look up any other file handle in the server's file system. This approach has the additional benefit that there is no need for a separate mount protocol. Instead, mounting can be integrated into the regular protocol for looking up files. A client can simply mount a remote file system by requesting the server to resolve names relative to the file system's root file handle using `putrootfh`.

Note 5.13 (Advanced: Automounting)

As we mentioned, the NFS naming model essentially provides users with their own name space. Sharing in this model may become difficult if users name the same file differently. One solution to this problem is to provide each user with a local name space that is partly standardized, and subsequently mounting remote file systems the same for each user.

Another problem with the NFS naming model has to do with deciding *when* a remote file system should be mounted. Consider a large system with thousands of users. Assume that each user has a local directory `/home` that is used to mount the home directories of other users. For example, Alice's home directory may be locally available to her as `/home/alice`, although the actual files are stored on a remote server. This directory can be automatically mounted when Alice logs into her workstation. In addition, she may have access to Bob's public files by accessing Bob's directory through `/home/bob`.

The question, however, is whether Bob's home directory should also be mounted automatically when Alice logs in. The benefit of this approach would be that the whole business of mounting file systems would be transparent to Alice. However, if this policy were followed for every user, logging in could incur a lot of communication and administrative overhead. In addition, it would require that all users are known in advance. A much better approach is to transparently mount another user's home directory on demand, that is, when it is first needed.

On-demand mounting of a remote file system (or actually an exported directory) is handled in NFS by an **automounter**, which runs as a separate process on the client's machine. The principle underlying an automounter is relatively simple. Consider a simple automounter implemented as a user-level NFS server on a Unix operating system. (For other implementations, see [Callaghan, 2000]).

Assume that for each user, the home directories of all users are available through the local directory `/home`, as described above. When a client machine boots, the automounter starts with mounting this directory. The effect of this local mount is that whenever a program attempts to access `/home`, the Unix kernel will forward a lookup operation to the NFS client, which in this case, will forward the request to the automounter in its role as NFS server, as shown in Figure 5.26.

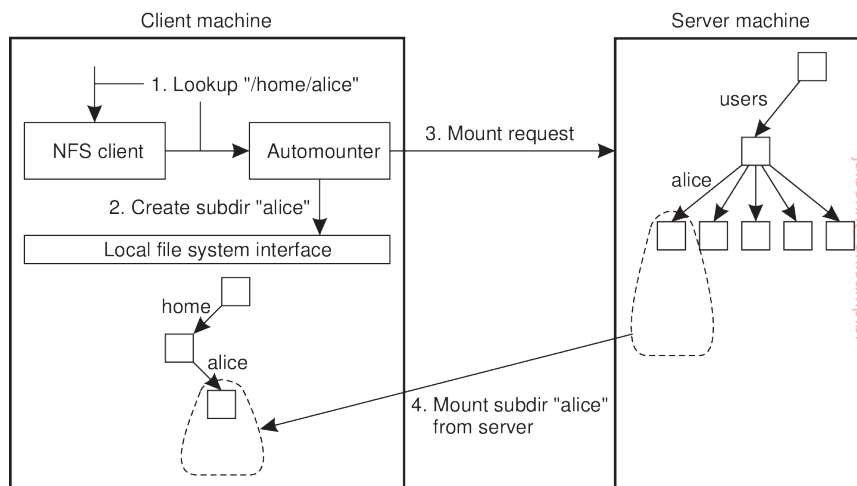


Figure 5.26: A simple automounter for NFS.

For example, suppose that Alice logs in. The login program will attempt to read the directory `/home/alice` to find information such as login scripts. The automounter will thus receive the request to look up subdirectory `/home/alice`, for which reason it first creates a subdirectory `/alice` in `/home`. It then looks up the NFS server that exports Alice's home directory to subsequently mount that directory in `/home/alice`. At that point, the login program can proceed.

The problem with this approach is that the automounter will have to be involved in all file operations to guarantee transparency. If a referenced file is not locally available because the corresponding file system has not yet been mounted, the automounter will have to know. In particular, it will need to handle all read and write requests, even for file systems that have already been mounted. This approach may incur a large performance problem. It would be better to have the automounter only mount/unmount directories, but otherwise stay out of the loop.

A simple solution is to let the automounter mount directories in a special subdirectory, and install a symbolic link to each mounted directory. This approach is shown in Figure 5.27.

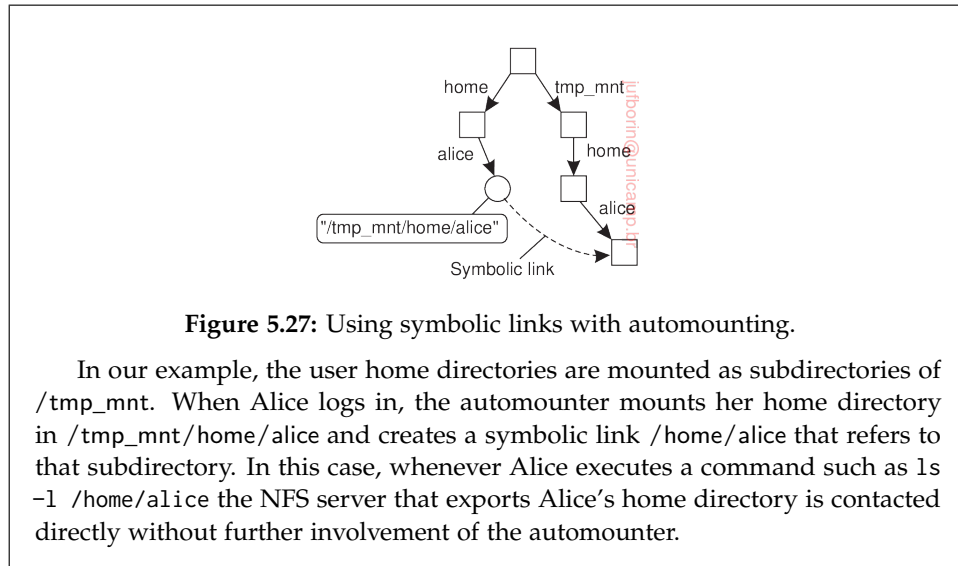


Figure 5.27: Using symbolic links with automounting.

In our example, the user home directories are mounted as subdirectories of `/tmp_mnt`. When Alice logs in, the automounter mounts her home directory in `/tmp_mnt/home/alice` and creates a symbolic link `/home/alice` that refers to that subdirectory. In this case, whenever Alice executes a command such as `ls -l /home/alice` the NFS server that exports Alice's home directory is contacted directly without further involvement of the automounter.

5.4 Attribute-based naming

Flat and structured names generally provide a unique and location-independent way of referring to entities. Moreover, structured names have been partly designed to provide a human-friendly way to name entities so that they can be conveniently accessed. In most cases, it is assumed that the name refers to only a single entity. However, location independence and human friendliness are not the only criterion for naming entities. In particular, as more information is being made available it becomes important to effectively search for entities. This approach requires that a user can provide merely a description of what he is looking for.

There are many ways in which descriptions can be provided, but a popular one in distributed systems is to describe an entity in terms of *(attribute, value)* pairs, generally referred to as **attribute-based naming**. In this approach, an entity is assumed to have an associated collection of attributes. Each attribute says something about that entity. By specifying which values a specific attribute should have, a user essentially constrains the set of entities that he is interested in. It is up to the naming system to return one or more entities that meet the user's description. In this section we take a closer look at attribute-based naming systems.

Directory services

Attribute-based naming systems are also known as **directory services**, whereas systems that support structured naming are generally called **naming systems**. With directory services, entities have a set of associated attributes that can be

used for searching. In some cases, the choice of attributes can be relatively simple. For example, in an e-mail system, messages can be tagged with attributes for the sender, recipient, subject, and so on. However, even in the case of e-mail, matters become difficult when other types of descriptors are needed, as is illustrated by the difficulty of developing filters that will allow only certain messages (based on their descriptors) to be passed through.

What it all boils down to is that designing an appropriate set of attributes is not trivial. In most cases, attribute design has to be done manually. Even if there is consensus on the set of attributes to use, practice shows that setting the values consistently by a diverse group of people is a problem by itself, as many will have experienced when accessing music and video databases on the Internet.

To alleviate some of these problems, research has been conducted on unifying the ways that resources can be described. In the context of distributed systems, one particularly relevant development is the **resource description framework (RDF)**. Fundamental to the RDF model is that resources are described as triplets consisting of a subject, a predicate, and an object. For example, (Person, name, Alice) describes a resource named Person whose name is Alice. In RDF, each subject, predicate, or object can be a resource itself. This means that Alice may be implemented as a reference to a file that can be subsequently retrieved. In the case of a predicate, such a resource could contain a textual description of that predicate. Resources associated with subjects and objects can be anything. References in RDF are essentially URLs.

If resource descriptions are stored, it becomes possible to query that storage in a way that is common for many attribute-based naming systems. For example, an application could ask for the information associated with a person named Alice. Such a query would return a reference to the person resource associated with Alice. This resource can then subsequently be fetched by the application.

In this example, the resource descriptions are stored at a central location. There is no reason why the resources should reside at the same location as well. However, not having the descriptions in the same place may incur a serious performance problem. Unlike structured naming systems, looking up values in an attribute-based naming system essentially requires an exhaustive search through all descriptors. (Various techniques can be applied to avoid such exhaustive searches, one obvious being indexing.) When considering performance, an exhaustive search may be less of a problem within a single, nondistributed data store, but simply sending a search query to hundreds of servers that jointly implement a distributed data store is generally not such a good idea. In the following, we will take a look at different approaches to solving this problem in distributed systems.

Hierarchical implementations: LDAP

A common approach to tackling distributed directory services is to combine structured naming with attribute-based naming. This approach has been widely adopted, for example, in Microsoft's Active Directory service and other systems. Many of these systems use, or rely on the **lightweight directory access protocol** commonly referred simply as **LDAP**. The LDAP directory service has been derived from OSI's X.500 directory service. As with many OSI services, the quality of their associated implementations hindered widespread use, and simplifications were needed to make it useful. Detailed information on LDAP can be found in Arkills [2003].

Conceptually, an LDAP directory service consists of a number of records, usually referred to as directory entries. A directory entry is comparable to a resource record in DNS. Each record is made up of a collection of (*attribute, value*) pairs, where each attribute has an associated type. A distinction is made between single-valued attributes and multiple-valued attributes. The latter typically represent arrays and lists. As an example, a simple directory entry identifying the network addresses of some general servers from Figure 5.23 is shown in Figure 5.28.

Attribute	Abbr.	Value
Country	C	NL
Locality	L	Amsterdam
Organization	O	VU University
OrganizationalUnit	OU	Computer Science
CommonName	CN	Main server
Mail_Servers	–	137.37.20.3, 130.37.24.6, 137.37.20.10
FTP_Server	–	130.37.20.20
WWW_Server	–	130.37.20.20

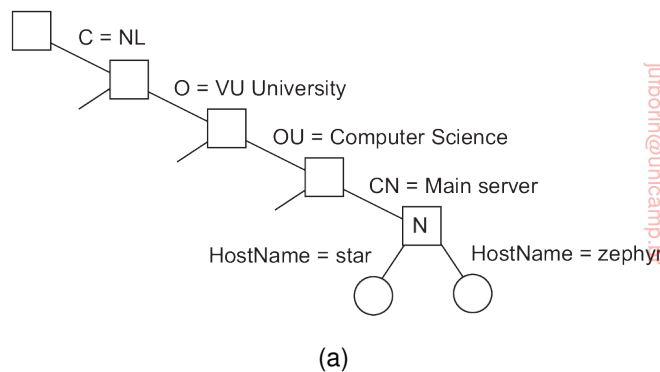
Figure 5.28: A simple example of an LDAP directory entry using LDAP naming conventions.

In our example, we have used a naming convention described in the LDAP standards, which applies to the first five attributes. The attributes Organization and OrganizationalUnit describe, respectively, the organization and the department associated with the data that are stored in the record. Likewise, the attributes Locality and Country provide additional information on where the entry is stored. The CommonName attribute is often used as an (ambiguous) name to identify an entry within a limited part of the directory. For example, the name "Main server" may be enough to find our example entry given the specific values for the other four attributes Country, Locality, Organization, and OrganizationalUnit. In our example, only attribute Mail_Servers has multiple values associated with it. All other attributes have only a single value.

The collection of all directory entries in an LDAP directory service is

called a **directory information base (DIB)**. An important aspect of a DIB is that each record is uniquely named so that it can be looked up. Such a globally unique name appears as a sequence of naming attributes in each record. Each naming attribute is called a **relative distinguished name**, or **RDN** for short. In our example in Figure 5.28 the first five attributes are all naming attributes. Using the conventional abbreviations for representing naming attributes in LDAP, as shown in Figure 5.28 the attributes Country, Organization, and OrganizationalUnit could be used to form the globally unique name $/C = NL/O = VU\ University/OU = Computer\ Science$. analogous to the DNS name `nl.vu.cs`.

As in DNS, the use of globally unique names by listing RDNs in sequence, leads to a hierarchy of the collection of directory entries, which is referred to as a **directory information tree (DIT)**. A DIT essentially forms the naming graph of an LDAP directory service in which each node represents a directory entry. In addition, a node may also act as a directory in the traditional sense, in that there may be several children for which the node acts as parent. To explain, consider the naming graph as partly shown in Figure 5.29. (Recall that labels are associated with edges.)



Attribute	Value	Attribute	Value
Locality	Amsterdam	Locality	Amsterdam
Organization	VUUniversity	Organization	VUUniversity
OrganizationalUnit	ComputerScience	OrganizationalUnit	ComputerScience
CommonName	Mainserver	CommonName	Mainserver
HostName	star	HostName	zephyr
HostAddress	192.31.231.42	HostAddress	137.37.20.10

(b)

Figure 5.29: (a) Part of a directory information tree. (b) Two directory entries having HostName as RDN.

Node N corresponds to the directory entry shown in Figure 5.28. At the same time, this node acts as a parent to a number of other directory entries

that have an additional naming attribute `HostName` that is used as an RDN. For example, such entries may be used to represent hosts as shown in Figure 5.29.

A node in an LDAP naming graph can thus simultaneously represent a directory in the traditional sense as we discussed previously, as well as an LDAP record. This distinction is supported by two different lookup operations. The `read` operation is used to read a single record given its path name in the DIT. In contrast, the `list` operation is used to list the names of all outgoing edges of a given node in the DIT. Each name corresponds to a child node of the given node. Note that the `list` operation does not return any records; it merely returns names. In other words, calling `read` with as input the name `/C = NL/O = VU University/OU = Computer Science/CN = Main server` will return the record shown in Figure 5.29, whereas calling `list` will return the names `star` and `zephyr` from the entries shown in Figure 5.29 as well as the names of other hosts that have been registered in a similar way.

Implementing an LDAP directory service proceeds in much the same way as implementing a naming service such as DNS, except that LDAP supports more lookup operations as we will discuss shortly. When dealing with a large-scale directory, the DIT is usually partitioned and distributed across several servers, known as **directory service agents (DSA)**. Each part of a partitioned DIT thus corresponds to a zone in DNS. Likewise, each DSA behaves very much the same as a normal name server, except that it implements a number of typical directory services, such as advanced search operations.

Clients are represented by what are called **directory user agents**, or simply **DUA**. A DUA is similar to a name resolver in structured-naming services. A DUA exchanges information with a DSA according to a standardized access protocol.

What makes an LDAP implementation different from a DNS implementation are the facilities for searching through a DIB. In particular, facilities are provided to search for a directory entry given a set of criteria that attributes of the searched entries should meet. For example, suppose that we want a list of all main servers at VU University. Using the notation defined in Howes [1997], such a list can be returned using a search operation like

```
search("(C=NL)(O=VU University)(OU=*)(CN=Main server)")
```

In this example, we have specified that the place to look for main servers is the organization named `VU_University` in country `NL`, but that we are not interested in a particular organizational unit. However, each returned result should have the `CN` attribute equal to `Main_server`.

As we already mentioned, searching in a directory service is generally an expensive operation. For example, to find all main servers at VU University requires searching all entries at each department and combining the results in a single answer. In other words, we will generally need to access several leaf nodes of a DIT in order to get an answer. In practice, this also means that several DSAs need to be accessed. In contrast, naming services can often be

implemented in such a way that a lookup operation requires accessing only a single leaf node.

This whole setup of LDAP can be taken one step further by allowing several trees to co-exist, while also being linked to each other. This approach is followed in Microsoft's Active Directory leading to a *forest* of LDAP domains [Allen and Lowe-Norris, 2003]. Obviously, searching in such an organization can be overwhelmingly complex. To circumvent some of the scalability problems, Active Directory usually assumes there is a global index server (called a global catalog) that can be searched first. The index will indicate which LDAP domains need to be searched further.

Although LDAP by itself already exploits hierarchy for scalability, it is common to combine LDAP with DNS. For example, every tree in LDAP needs to be accessible at the root (known in Active Directory as a domain controller). The root is often known under a DNS name, which, in turn, can be found through an appropriate SRV record as we explained above.

Decentralized implementations

Notably with the advent of peer-to-peer systems, researchers have also been looking for solutions for decentralized attribute-based naming systems. In particular, peer-to-peer systems are often used to store files. Initially, files could not be searched—they could only be looked up by their key. However, having the possibility to search for a file based on descriptors can be extremely convenient, where each descriptor is nothing but an *(attribute, value)* pair. Obviously, querying every node in a peer-to-peer system to see if it contains a file matching one or more of such pairs is infeasible. What we need is a mapping of *(attribute, value)* pairs to **index servers**, which, in turn, point to files matching those pairs.

Using a distributed index

Let us first look at the situation of building a (distributed) index. The basic idea is that a search query is formulated as a list of *(attribute, value)* pairs, just as in the case of our LDAP examples. The result should be a list of (references to) entities that match *all* pairs. In the case of a peer-to-peer system storing files, a list of keys to relevant files may be returned, after which the client can look up each of those files using the returned keys.

A straightforward approach toward a distributed index is the following. Assume there are d different attributes. In that case we can use a server for each of the d attributes, where a server for attribute A maintains a set of (E, val) pairs for each entity E that has the value val for attribute A . A search query such as

```
search(("(Country=NL)(Organization=VU University)
      (OrganizationalUnit=*)(CommonName=Main server)"))
```

would be sent to the servers for Country, Organization, and CommonName, respectively, after which the client would need to see which entities occur in *all* three sets as returned by the servers. To prevent that a server needs to maintain a very large set of entities, the set for each server can be further partitioned and distributed across several subservers, each subserver associated with the same attribute.

More precisely, if we have a set of attributes $\{a^1, \dots, a^N\}$, then for each attribute a^k we associate a set $\mathbf{S}^k = \{S_1^k, \dots, S_{n_k}^k\}$ of n_k servers. Assuming that an attribute a^k takes values from a set R^k , we construct a global mapping F such that

$$F(a^k, v) = S_j^k \text{ with } S_j^k \in \mathbf{S}^k \text{ and } v \in R^k$$

In this example, server S_j^k would keep track of each key associated with a file having $a^k = v$. The beauty of this scheme is its simplicity. If $L(a^k, v)$ is the set of keys returned by server $F(a^k, v)$, then a query can be formulated as a logical expression such as

$$(F(a^1, v^1) \wedge F(a^2, v^2)) \vee F(a^3, v^3)$$

which can then be processed on the client side by constructing the set

$$(L(a^1, v^1) \cap L(a^2, v^2)) \cup L(a^3, v^3)$$

Unfortunately, there are important drawbacks to this scheme. First, any query involving k attributes requires contacting k index servers, which may incur significant communication costs. Furthermore, and related, is that the client is required to process the sets returned by the index servers. Just imagine that each file has two attributes `firstName` and `lastName`, respectively, and that a client is looking for the file owned by Pheriby Smith. Now, although Pheriby may be quite unique for a first name, Smith definitely is not. However, our poor client will have to receive perhaps millions of keys of files for which `lastName = Smith`, while there may actually be only a handful of files for which `firstName = Pheriby`. Thirdly, although this scheme does allow to leave certain attributes unspecified (by simply not mentioning them in the query), it does not easily support range queries, such as, `price = [1000 – 2500]`.

Space-filling curves

A common approach to implementing decentralized attribute-based naming systems is to use what are known as **space-filling curves**. The basic idea is to map the N -dimensional space covered by the N attributes $\{a^1, \dots, a^N\}$ into a single dimension, and then use, for example, a simple hashing technique to distribute the resultant space among index servers. One of the key issues is to have *(attribute, value)* pairs that are “close” to each other be handled by the same index server.

Let us make matters concrete and look into one popular case, namely Hilbert space-filling curves (see, for example, Lawder and King [2000]). These are easiest to explain by looking only at two dimensions, that is, considering only two distinct attributes. The possible values that each attribute can have corresponds to one axis in a two-dimensional space. Without loss of generality, we assume that each attribute takes on values in the interval $[0, 1)$. As a first approximation of the square, we divide it into four quadrants, as shown in Figure 5.30(a). All data values (x, y) with $0 \leq x, y < 0.5$ are associated with index 0. Values (x, y) with $0.5 \leq x, y < 1.0$ are associated with index 2.

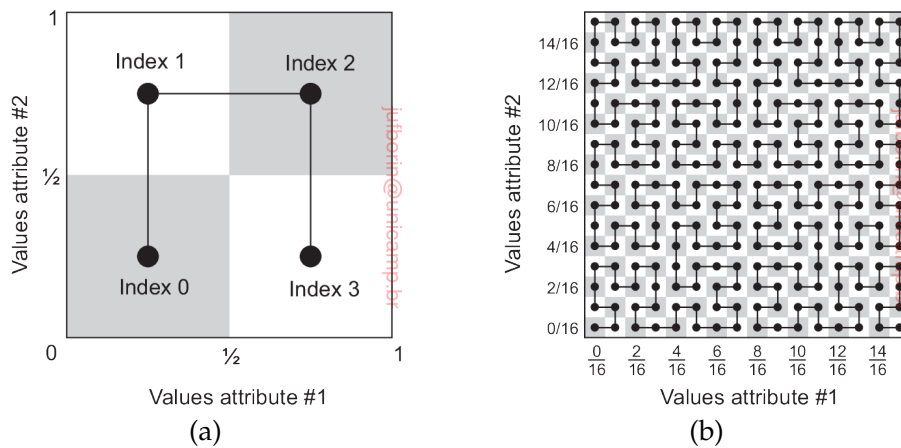


Figure 5.30: Reducing a two-dimensional space to a single dimension through a Hilbert space-filling curve of (a) order 1, and (b) order 4.

We can repeat this procedure recursively for each subsquare: divide it into four smaller squares and connect the smaller squares through a single line. Using rotation and reflection, we make sure that this line can be nicely connected to the one in the previously neighboring larger subsquare (which has also been divided into smaller squares). To illustrate, where Figure 5.30(a) shows a Hilbert curve of order 1, Figure 5.30(b) shows a curve of order 4 with 256 indices. In general, a Hilbert curve of order k connects 2^{2k} subsquares, and thus has also 2^{2k} indices. There are various ways in which we can systematically draw a curve through a two-dimensional space that has been partitioned into equally sized squares. Furthermore, the process can be easily expanded to higher dimensions, as explained by Sagan [1994] and Bader [2013].

An important property of space-filling curves is that they preserve locality: two indices that are close to each other on the curve correspond to two points that are also close to each other in the multidimensional space. (Note that the reverse is not always true: two points close to each other in the multidimensional space need not necessarily lie close to each other on the

curve.)

To complete the story, several things need to be done. First, attribute values need to be indexed. Assume that we are dealing with a total of N possible attributes $\{a^1, \dots, a^N\}$, and that each entity assigns a value to each of these N attributes (possibly the equivalent of a “don’t care” value). To keep matters simple, we assume that each attribute value is normalized to a value in the interval $[0, 1)$. Then clearly, an entity E having the tuple of values $(v^1, \dots, v^N)$ is associated with a real-valued coordinate in an N -dimensional space, in turn uniquely associated with an N -dimensional subsquare as we discussed for the two-dimensional case. The center of such a subsquare corresponds to an index on the associated Hilbert space-filling curve, and is now the index associated with entity E . Of course, multiple entities whose associated coordinates fall in the same subsquare will all have the same index. To avoid such collisions as much as possible, we need to use high-ordered space-filling curves. Orders of 32 or 64 are not uncommon.

Second, we also need to be able to search for entities. The principle of searching for entities based on their attribute values should now be clear. Suppose we were looking for files whose two attribute values a^1 and a^2 lie in intervals $[v_l^1, v_u^1)$ and $[v_l^2, v_u^2)$, respectively (with $v_l^i < v_u^i$). Clearly, this delineates a rectangular region through which the curve passes, and all files indexed by those segments of the curve that intersect with that region match the search criterion. We therefore need an operation that returns a series of curve-related indices given a region (expressed in terms of subsquares) in the associated N -dimensional space. Such an operation is clearly dependent on which space-filling curve has been used, but interestingly, need not be dependent on actual entities.

Finally, we need to maintain (references to) the entities associated with indices. One approach, used in the Squid system [Schmidt and Parashar, 2008], is to use a Chord ring. In Squid, the index space is chosen to be the same as that of the Chord ring, that is, both use m -bit identifiers. Then clearly, the Chord node responsible for index i will store (references to) the entities indexed by i .

Note 5.14 (More information: Other examples)

Decentralized implementations of attribute-based naming systems have received a lot of attention. The ones based on space-filling curves are relatively popular, but several alternatives have been proposed as well. Let us take a look at some representative examples to shed some light on the design space.

Attribute-value trees Here is another example where *(attribute, value)* pairs are supported by a DHT-based system. First, assume that queries consist of a conjunction of pairs as with LDAP, that is, a user specifies a list of attributes, along with the unique value he wants to see for every attribute. Such single-

valued queries are supported in the INS/Twine system [Balazinska et al., 2002]. Each entity (referred to as a resource) is assumed to be described by means of possibly hierarchically organized attributes such as shown in Figure 5.31. Each such description is translated into an **attribute-value tree (AVTree)** which is then used as the basis for an encoding that maps well onto a DHT-based system.



Figure 5.31: (a) A general description of a resource. (b) Its representation as an AVTree.

The main issue is to transform the AVTrees again into a collection of keys that can be looked up in a DHT system. In this case, every path originating in the root is assigned a unique hash value, where a path description starts with a link (representing an attribute), and ends either in a node (value), or another link. Taking Figure 5.31 as our example, the following keys of all such paths are considered:

Key	Computed as
h_1 :	$hash(type-book)$
h_2 :	$hash(type-book-author)$
h_3 :	$hash(type-book-author-Tolkien)$
h_4 :	$hash(type-book-title)$
h_5 :	$hash(type-book-title-LOTR)$
h_6 :	$hash(genre-fantasy)$

A node responsible for hash value h_i will keep (a reference to) the actual resource. In our example, this may lead to six nodes storing information on Tolkien's Lord of the Rings. However, the benefit of this redundancy is that it will allow supporting partial queries. For example, consider a query such as "Return books written by Tolkien." This query is translated into the AVTree shown in Figure 5.32 leading to computing the following three hashes:

h_1 : $hash(type-book)$
 h_2 : $hash(type-book-author)$
 h_3 : $hash(type-book-author-Tolkien)$

These values will be sent to nodes that store information on Tolkien's books, and will at least return Lord of the Rings. Note that a hash such as h_1 is rather general and will be generated often. These type of hashes can be filtered out of the system. Moreover, it is not difficult to see that only the most specific hashes need to be evaluated.

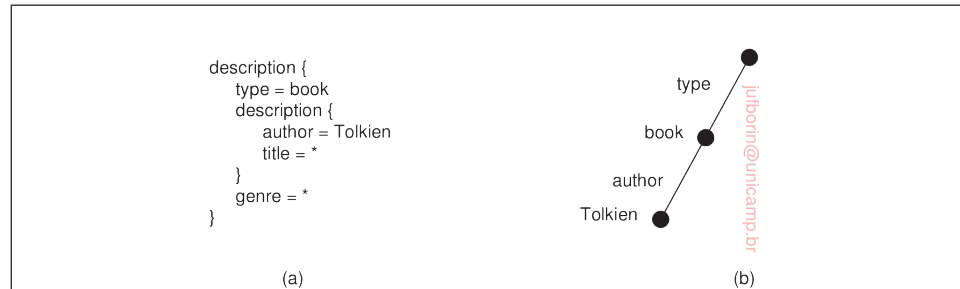


Figure 5.32: (a) The resource description of a query. (b) Its representation as an AVTree.

SWORD: including range queries Let us take a look at queries that can contain range specifications for attribute values. We discuss a solution adopted in the SWORD resource discovery system [Oppenheimer et al., 2005].

In SWORD, *(attribute, value)* pairs are first transformed into a key for a DHT. These pairs always contain a single value; only queries may contain value ranges for attributes. When computing the key (by means of a hash) the name of the attribute and its value are kept separate. Specific bits in the key will identify the attribute name, while others identify its value. In addition, the key will contain a number of random bits to guarantee uniqueness among all keys that need to be generated. In this way, the space of attributes is conveniently partitioned: if n bits are reserved to code attribute names, 2^n different server groups can be used, one group for each attribute name. Likewise, by using m bits to encode values, a further partitioning per server group can be applied to store specific *(attribute, value)* pairs. DHTs are used only for distributing attribute names.

For each attribute name, the possible range of its value is partitioned into subranges and a single server is assigned to each subrange. To explain, consider a resource description with two attributes: a^1 taking values in the range $[1..10]$ and a^2 taking values in the range $[101..200]$. Assume there are two servers for a^1 : S^{11} takes care of recording values of a^1 in $[1..5]$, and S^{12} for values in $[6..10]$. Likewise, server S^{21} records values for a^2 in range $[101..150]$ and server S^{22} for values in $[151..200]$. Then, when an entity E has associated attribute values $(a^1 = 7, a^2 = 175)$, server S^{12} and server S^{22} will maintain a copy of, or a reference to E .

The advantage of this scheme is that range queries can be easily supported. When a query is issued to return resources that have a^2 lying between 165 and 189, the query can be forwarded to server S^{22} who can then return the resources that match the query range. The drawback, however, is that updates need to be sent to multiple servers. Moreover, it is not immediately clear how well the load is balanced between the various servers. In particular, if certain range queries turn out to be very popular, specific servers will receive a high fraction of all queries.

Summarizing remarks There are indeed many different ways of supporting attribute-based naming systems in a decentralized fashion. The essence in all

cases is to assign attributes to servers so that clients know where to direct their queries, yet at the same time make sure that there is a balance in the load for the set of servers. In this light, supporting range queries requires special attention, if only to decide, as in SWORD, which server is going to be responsible for which subrange.

In practice, we see that when dealing with N attributes many systems model the collection of *(attribute, value)* pairs as an N -dimensional space in which each entity is represented by a unique point in that space. Conceptually, a search addresses a subspace and leads to identifying the servers responsible for that subspace. In the simplest case, we assign each attribute to one server leading to $\mathcal{O}(N)$ servers. In this scheme, a query addressing k attributes needs to be sent to k servers, while the querying client needs to combine the results. We have discussed this case previously. The problem is to divide the ranges per attribute among subservers such that we have a reasonable balance of the workload. A solution to this problem is discussed in [Bharambe et al., 2004].

Instead of letting a client combine results, we can let servers collaborate. To this end, the N -dimensional space is divided into subspaces by splitting each dimension d into n_d intervals. This splitting leads to a total of $n_1 \times \cdots \times n_N$ subspaces where each subspace is assigned to a separate server. Even with $n_d = 2$ for each dimension, we will face a total of $\mathcal{O}(2^N)$ servers. Using space-filling curves, we can reduce the number of dimensions to one, and use a separate technique for deciding which N -dimensional subspace is served by which server. Practice indicates that load balancing may become an issue. An alternative solution in which the number of dimensions is still reduced, but larger than one, while also maintaining load balancing, has been built into HyperDex [Escriva et al., 2012]. The authors also address the problem of replication and consistency in case the naming system at the same time stores the entities which it indexes. In that case, whenever a server indexes an entity E , it will have to be copied to the respective server.

We finally point to an attribute-based naming system that integrates naming with resource selection service [Stratan et al., 2012]. Again, the attribute space is modeled by an N -dimensional space in which each resource is associated with a coordinate. In this situation, each resource maintains a link to another resource, but one that is responsible for a subspace of exponentially increasing size. The net effect is that each resource needs to have only a fixed number of neighbors, while routing a query to the relevant subspace takes only a linear number of steps. The organization is akin to the use of finger tables in Chord.

5.5 Summary

Names are used to refer to entities. Essentially, there are three types of names. An address is the name of an access point associated with an entity, also simply called the address of an entity. An identifier is another type of name. It has three properties: each entity is referred to by exactly one identifier, an identifier refers to only one entity, and is never assigned to another entity.

Finally, human-friendly names are targeted to be used by humans and as such are represented as character strings. Given these types, we make a distinction between flat naming, structured naming, and attribute-based naming.

Systems for flat naming essentially need to resolve an identifier to the address of its associated entity. This locating of an entity can be done in different ways. The first approach is to use broadcasting or multicasting. The identifier of the entity is broadcast to every process in the distributed system. The process offering an access point for the entity responds by providing an address for that access point. Obviously, this approach has limited scalability.

A second approach is to use forwarding pointers. Each time an entity moves to a next location, it leaves behind a pointer telling where it will be next. Locating the entity requires traversing the path of forwarding pointers. To avoid large chains of pointers, it is important to reduce chains periodically.

A third approach is to allocate a home to an entity. Each time an entity moves to another location, it informs its home where it is. Locating an entity proceeds by first asking its home for the current location.

A fourth approach is to organize all nodes into a structured peer-to-peer system, and systematically assign nodes to entities taking their respective identifiers into account. By subsequently devising a routing algorithm by which lookup requests are moved toward the node responsible for a given entity, efficient and robust name resolution is possible.

A fifth approach is to build a hierarchical search tree. The network is divided into nonoverlapping domains. Domains can be grouped into higher-level (nonoverlapping) domains, and so on. There is a single top-level domain that covers the entire network. Each domain at every level has an associated directory node. If an entity is located in a domain D , the directory node of the next higher-level domain will have a pointer to D . A lowest-level directory node stores the address of the entity. The top-level directory node knows about all entities.

Structured names are easily organized in a name space. A name space can be represented by a naming graph in which a node represents a named entity and the label on an edge represents the name under which that entity is known. A node having multiple outgoing edges represents a collection of entities and is also known as a context node or directory. Large-scale naming graphs are often organized as rooted acyclic directed graphs.

Naming graphs are convenient to organize human-friendly names in a structured way. An entity can be referred to by a path name. Name resolution is the process of traversing the naming graph by looking up the components of a path name, one at a time. A large-scale naming graph is implemented by distributing its nodes across multiple name servers. When resolving a path name by traversing the naming graph, name resolution continues at the next name server as soon as a node is reached implemented by that server.

More problematic are attribute-based naming schemes in which entities are

described by a collection of (*attribute, value*) pairs. Queries are also formulated as such pairs, essentially requiring an exhaustive search through all descriptors. Such a search is feasible only when the descriptors are stored in a single database. However, alternative solutions have been devised by which the pairs are mapped onto DHT-based systems, essentially leading to a distribution of the collection of entity descriptors. Using space-filling curves, we can then make different nodes responsible for different values of an attribute which helps in effectively distributing the load among the nodes in the case of search operations.